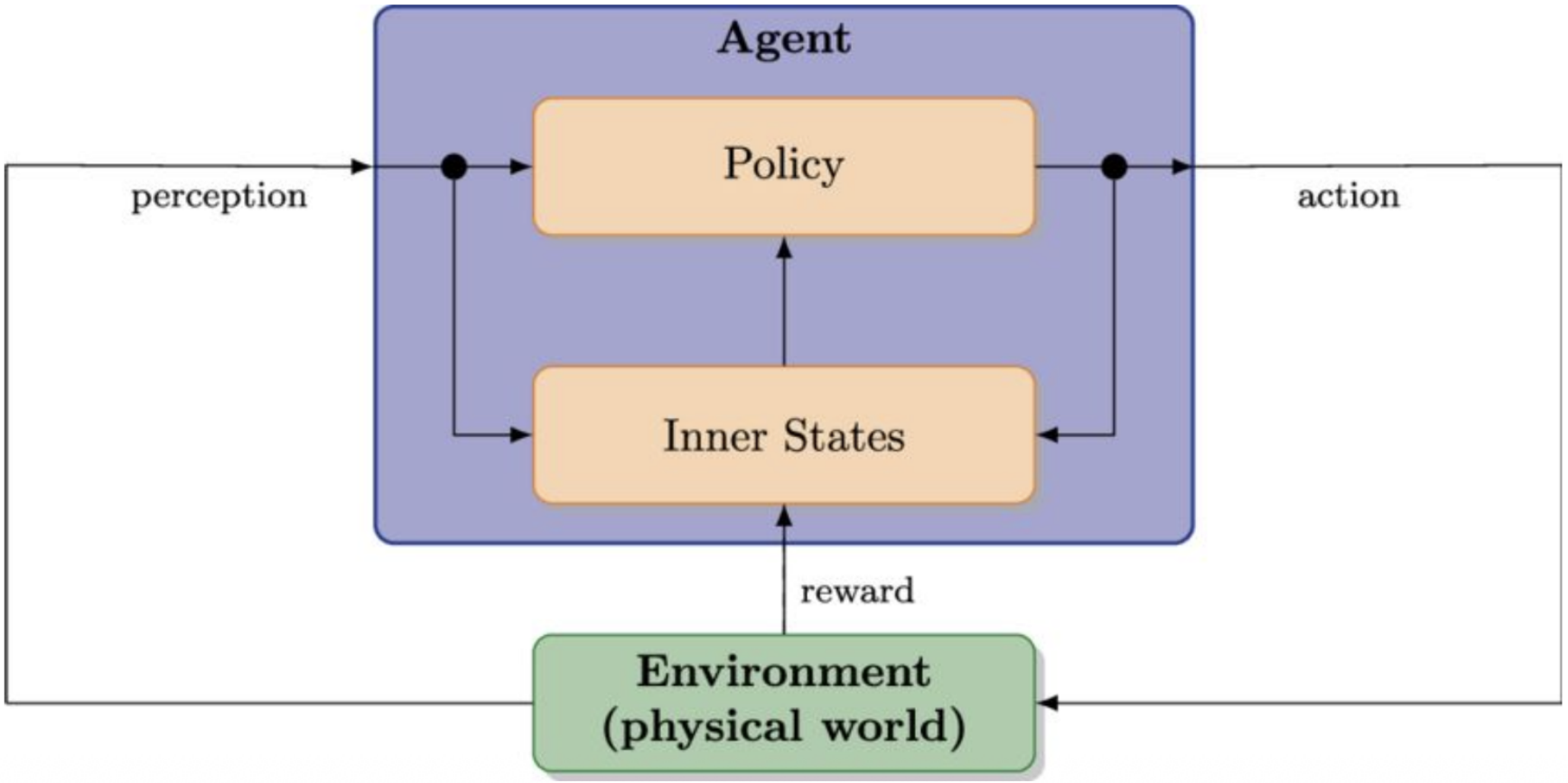
# Agents

Ashish Bansal

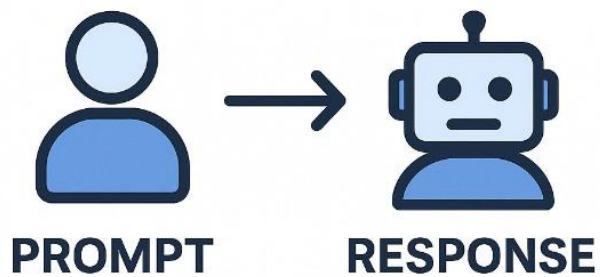
# 1980's Definition of an Agent (ex: RL Agent)

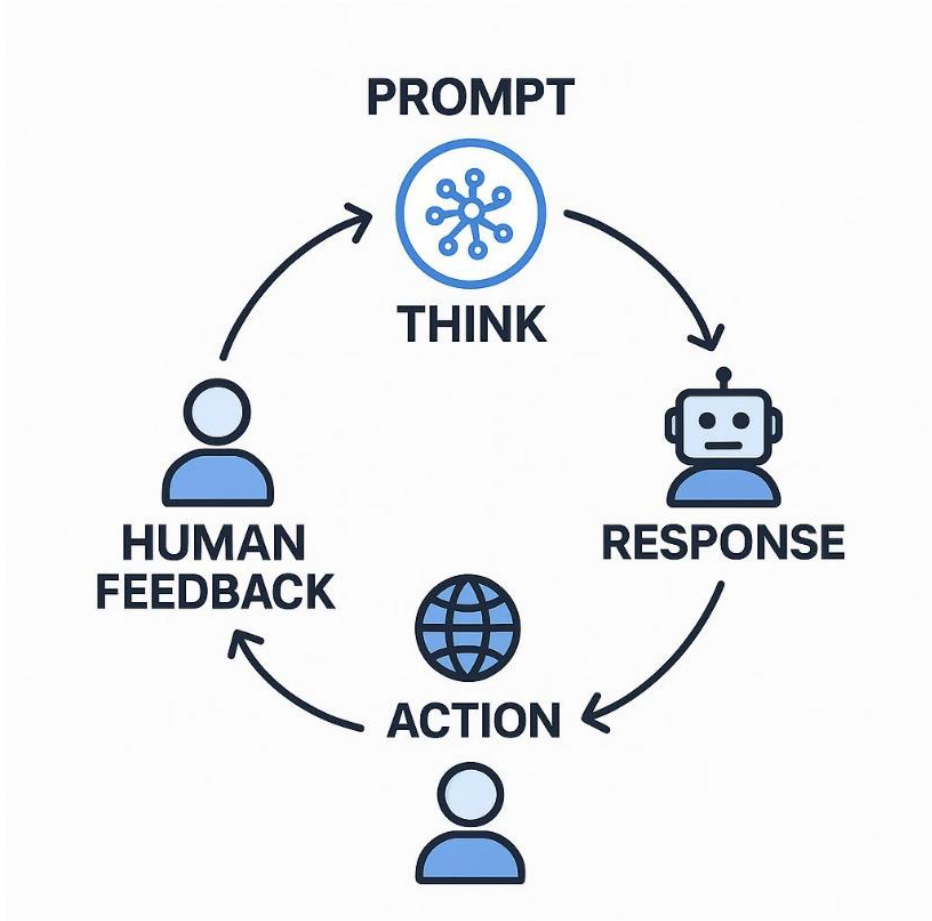


# AI Agents


think
interact
remember
AI Models
are good at understanding data
Agents
are good at interacting with environment and learning from them
AI Agents
can think, interact, remember, and answer

# AI Models vs. AI Agents



# AI Models vs. AI Agents: Simple Perspective

Human Brain

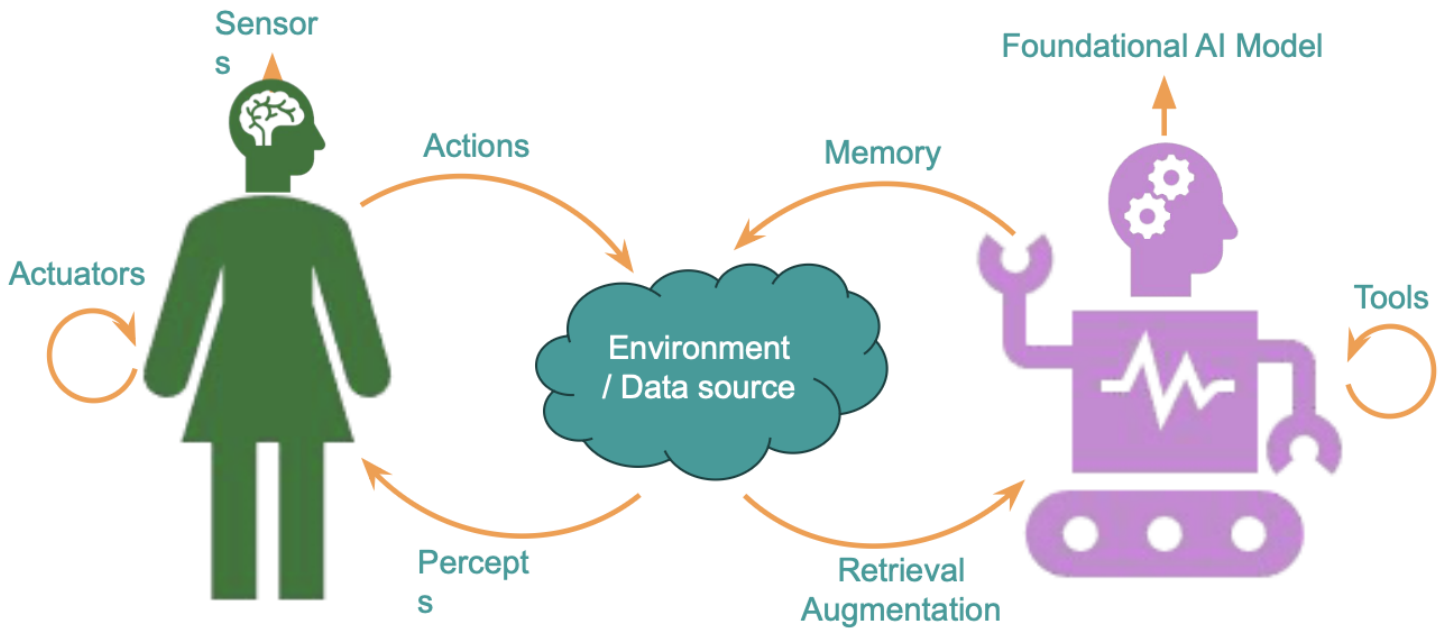
Foundational AI Model



*AI models mimic the human brain/ intelligence*

Human Agent

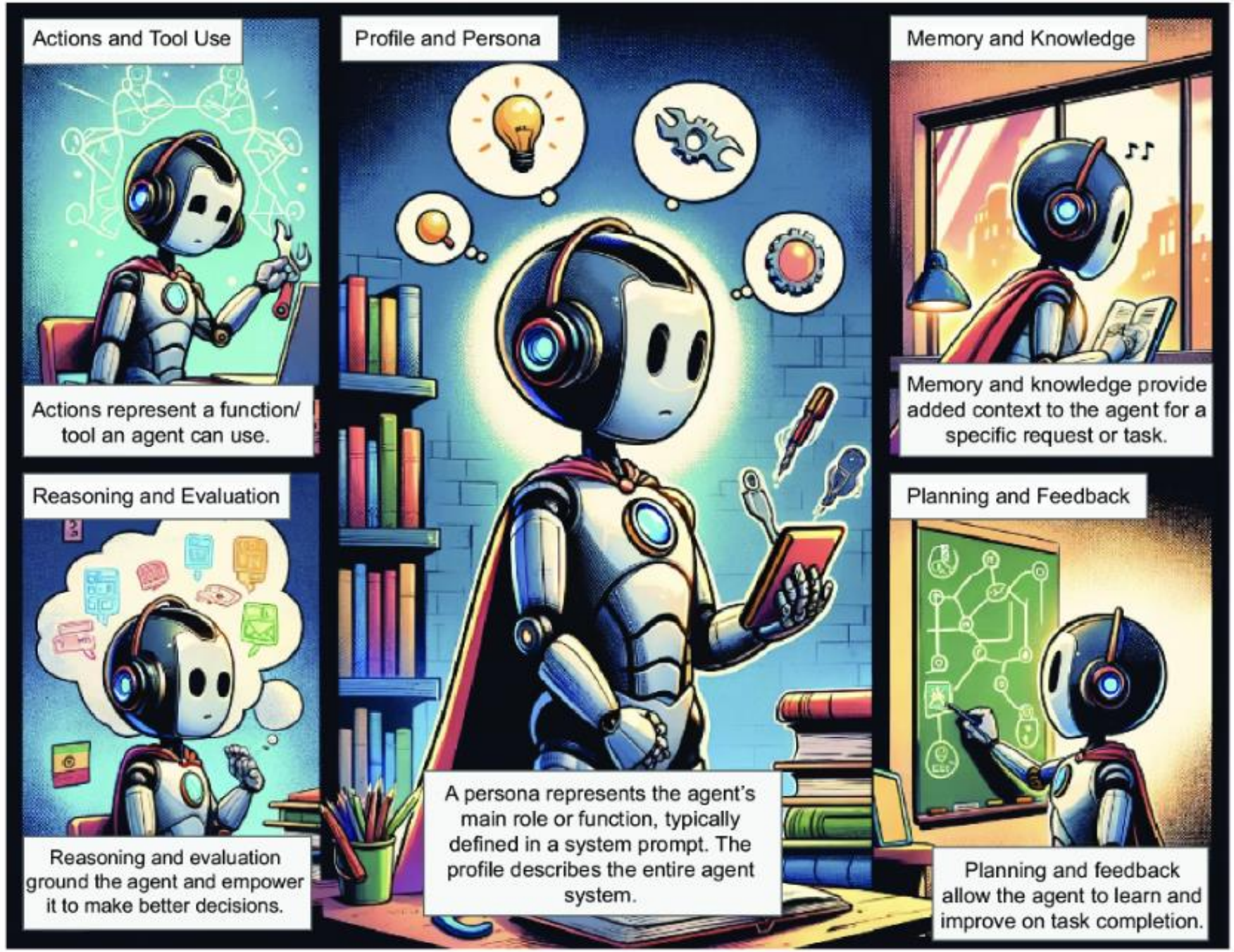
AI Agent




*LLM based AI agent mimics the entire human capability.*
*The core of an AI agent is an LLM model as the brain.*

# Tools , Environment , System prompt

| | Environment | Tools | System Prompt |
|---|---|---|---|
| **Coding** |  | bash, grep, read, write, ... | "You are an interactive coding assistant..." |
| **Computer use** |  | Click, type, screenshot, ... | "You are utilising an Ubuntu virtual machine with internet access..." |
| **Search** |  | Search_web, search_local, fetch... | "You are a search assistant..." |


Actions and Tool Use
Actions represent a function/ tool an agent can use.
Profile and Persona
A persona represents the agent's main role or function, typically defined in a system prompt. The profile describes the entire agent system.
Memory and Knowledge
Memory and knowledge provide added context to the agent for a specific request or task.
Reasoning and Evaluation
Reasoning and evaluation ground the agent and empower it to make better decisions.
Planning and Feedback
Planning and feedback allow the agent to learn and improve on task completion.



Profile and Persona



**Profile Contents**

Persona: Role, i.e., coder or tester
Demographics: Sex, age, background

**Profile Generation**

Handcrafted: Manually designed by humans
LLM generated: Directed by human prompts
Data generated: Constructed from data personas

**Agent persona: We'll understand how to clearly define the persona, specifying their role and characteristics to guide the agent effectively.**

**Agent role and demographics: We'll see how relevant demographic and role details can provide agent context, such as age, gender, or background, for a more relevant interaction.**

**Human vs. AI assistance for persona generation: We'll highlight the role of human involvement in persona generation, whether it's entirely human driven or assisted by LLMs or other agents.**

**Innovative persona techniques: Prompts generated through data or other novel approaches such as evolutionary algorithms to enhance agent capabilities.**

**Action targets: We'll learn the importance of defining action targets, whether for task completion, exploration, or communication, to clarify the agent's objectives.**

**Action space and impact: We'll learn the significance of understanding how actions affect task completion and their effect on the agent's environment, internal states, and self-knowledge.**

**Action generation methods: We'll see the various ways actions can be generated, such as manual creation, memory recollection, or plan following, to illustrate the diversity of agent behaviors.**

**Action and Tool Use**



**Action Target**
Semantic or native functions

**Action Space**
Tools, self-knowledge, other agents

**Action Impact**
Environments, new actions, internal states, other agents

**Action Generation**
Manual, memory recollection, plan following





**Memory and Knowledge**



**Retrieval Structure**
- Unified
- Hybrid

**Retrieval Formats**
- Language
- Databases
- Embeddings
- Lists

**Retrieval Operation**
- Augmentation
- Semantic Extraction
- Compression

**Retrieval structure variety: We'll learn about the diverse memory structures agents can employ, including unified and hybrid approaches, enabling flexibility in information storage.**

**Retrieval formats: We'll explore the various data sources for memory, such as language (e.g., PDF documents), databases (relational, object, or document), and embeddings, offering a rich pool of information to draw upon.**

**Semantic similarity: We'll learn how embeddings enable semantic similarity searches, facilitating efficient retrieval of relevant data and enhancing the agent's decision-making capabilities.**



## Reasoning and Evaluation



**Reasoning**
- Zero-shot prompting
- One-shot prompting
- Few-shot prompting
- Chain of thought
- Tree of thought
- Skeleton of thought

**Evaluation**
- Self-consistency
- Prompt chaining

**Reasoning enables the agent to self-reflect and internally reason out the completion of a task or tasks.**

**Evaluation provides the basis for an agent's self-reflection on working through and upon task completion.**

**We'll look at various planning strategies with and without feedback—from basic and sequential planners to automatic tool use with reasoning.**

**Feedback may come from a variety of sources, such as environmental, human, and an LLM via various constructive feedback patterns.**

## Planning and Feedback

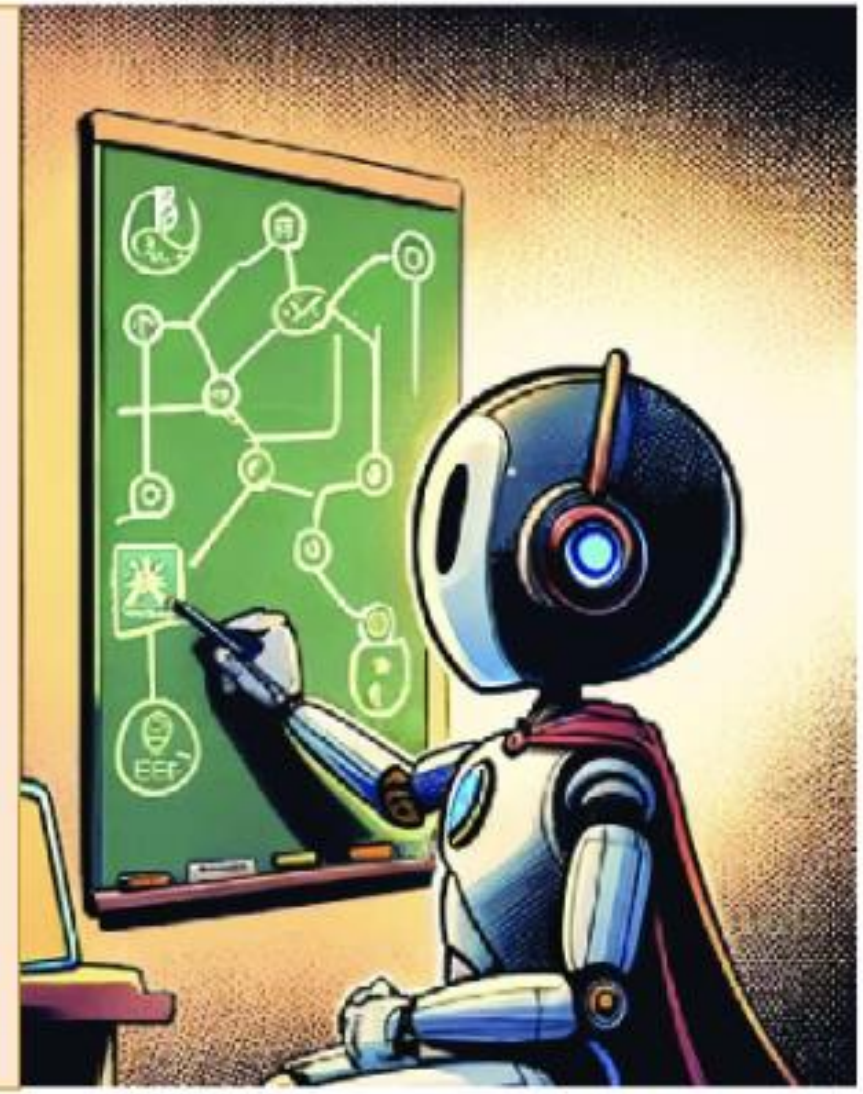


**Planning without feedback (autonomous)**

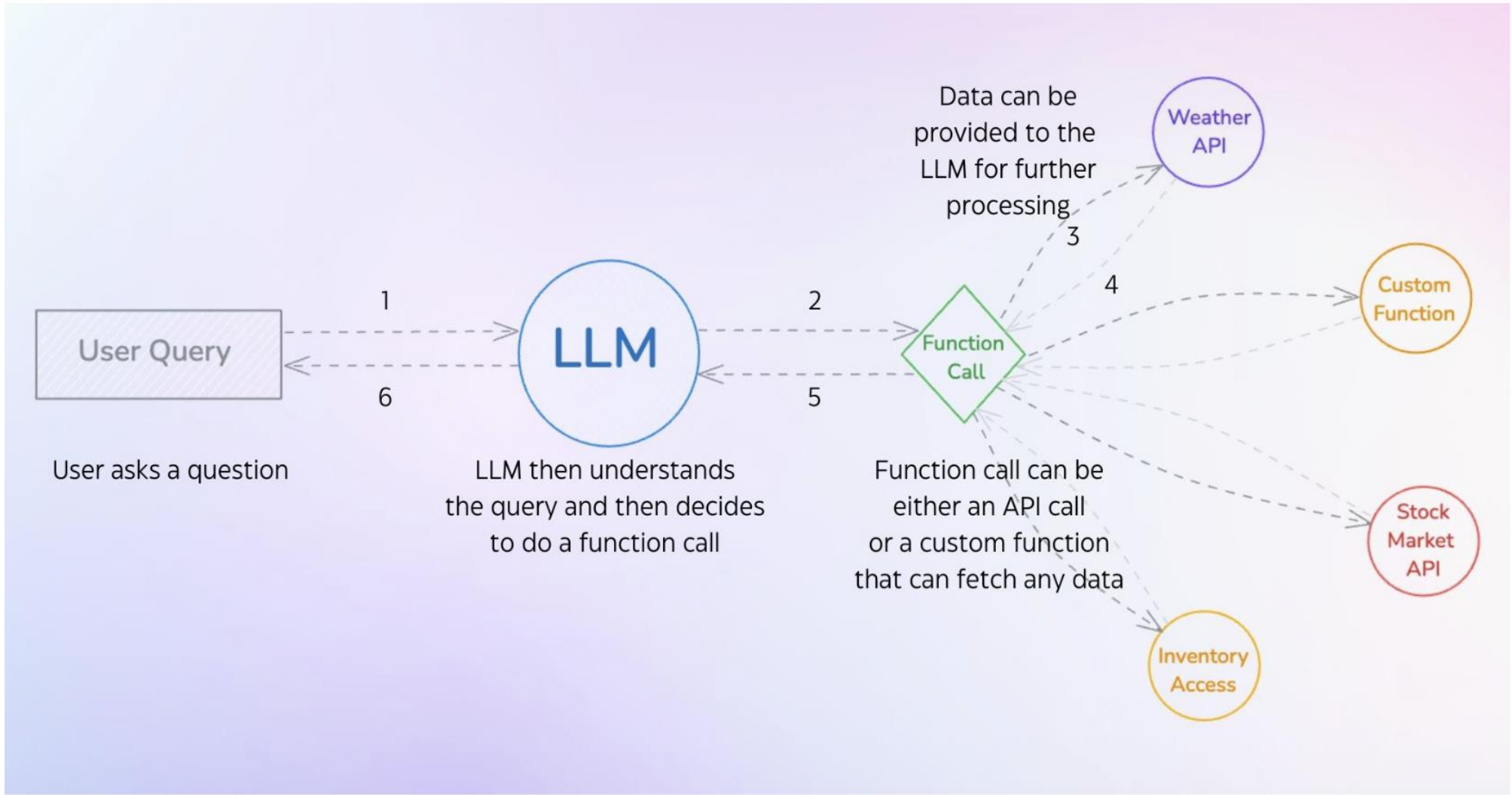
- Basic planning
- Automatic reasoning with tool use
- Sequential planning

**Planning with feedback**

- Environmental feedback
- Human feedback
- LLM feedback
- Adaptive constructive feedback



# Definition #1: AI Agents have *tools*



## Available tools

Here's an overview of the tools available in the OpenAI platform—select one of them for further guidance on usage.

### Function calling
Call custom code to give the model access to additional data and capabilities.

### Web search
Include data from the Internet in model response generation.

### Remote MCP servers
Give the model access to new capabilities via Model Context Protocol (MCP) servers.

### Skills
Upload and reuse versioned skill bundles in hosted shell environments.

### Shell
Run shell commands in hosted containers or in your own local runtime.

### Computer use
Create agentic workflows that enable a model to control a computer interface.

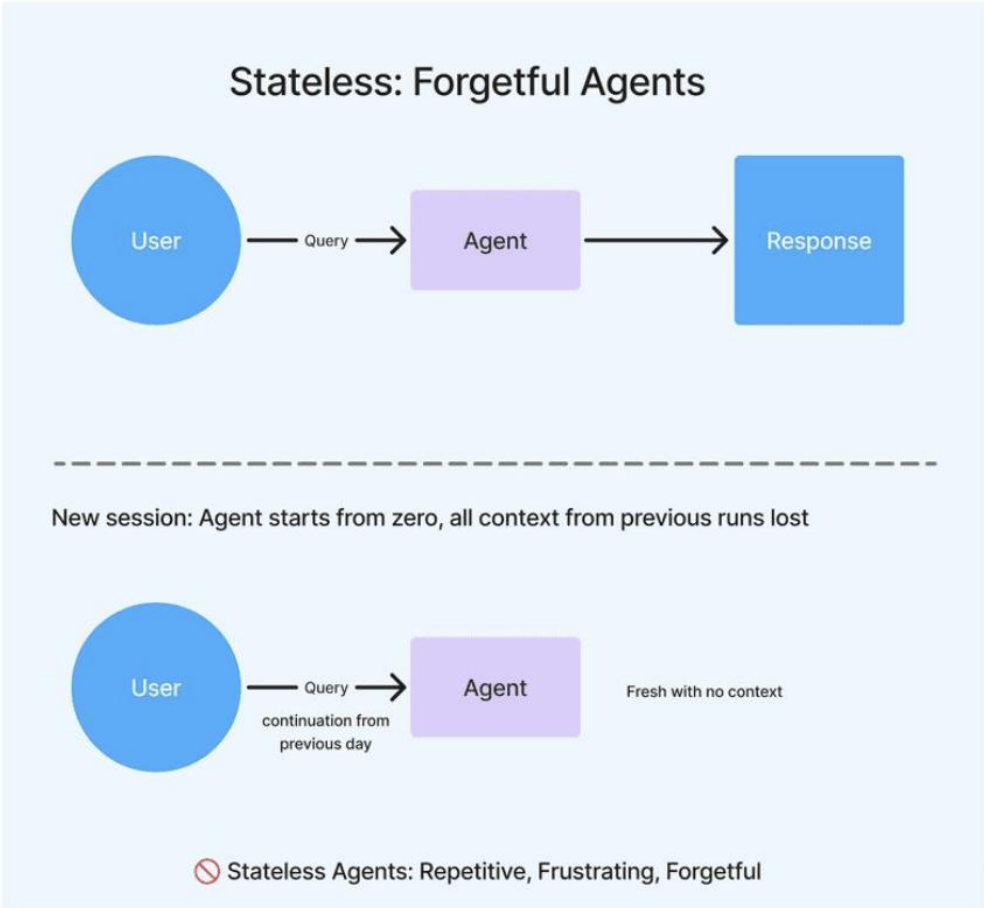
### Image generation
Generate or edit images using GPT Image.

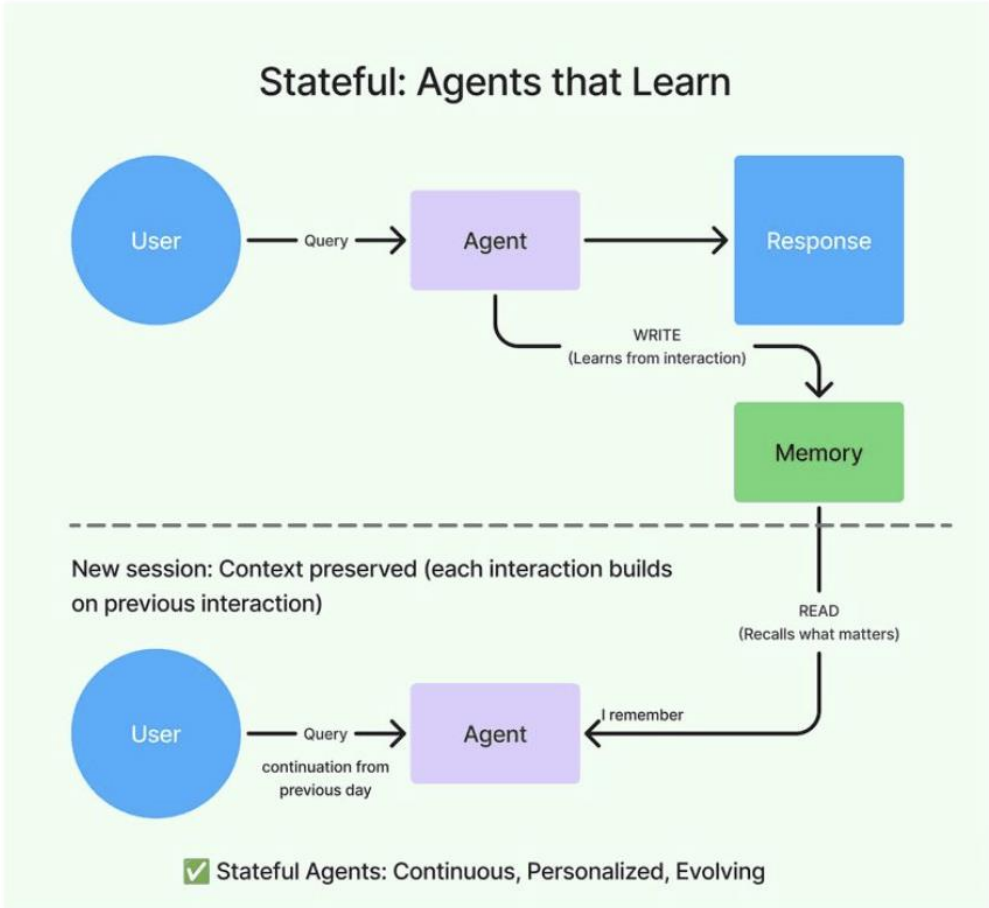
### File search
Search the contents of uploaded files for context when generating a response.

### Tool search
Dynamically load relevant tools into the model's context to optimize token usage.

### Programmatic Tool Calling
Let models compose and run JavaScript that orchestrates tool calls.

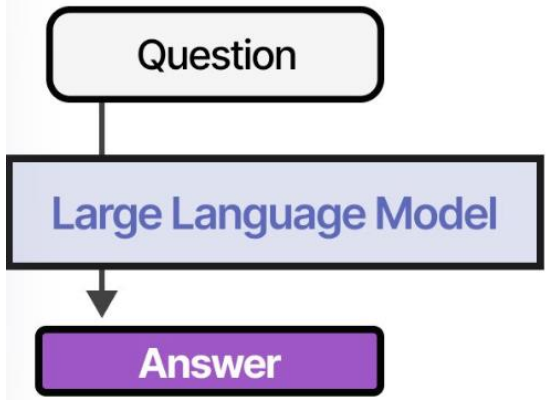
# Definition #2: AI Agents have memory



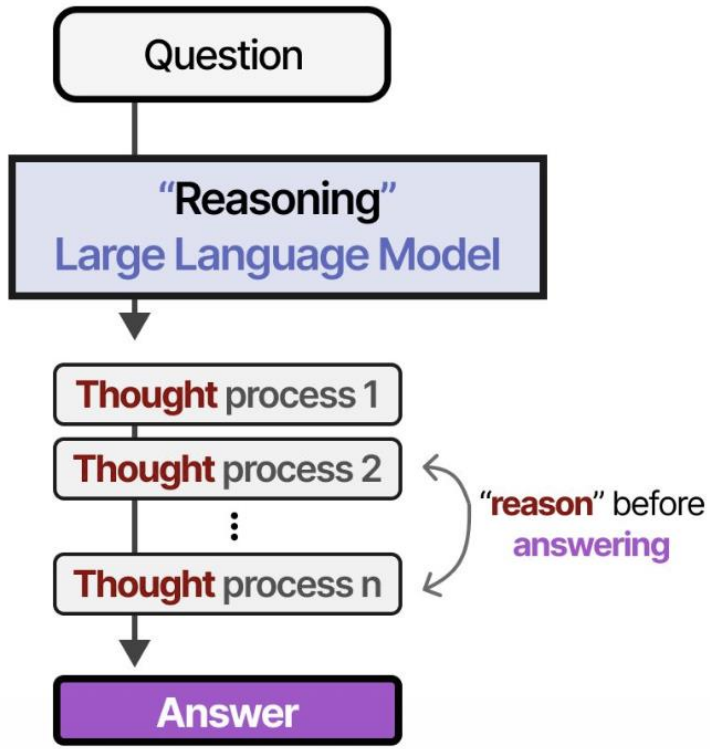



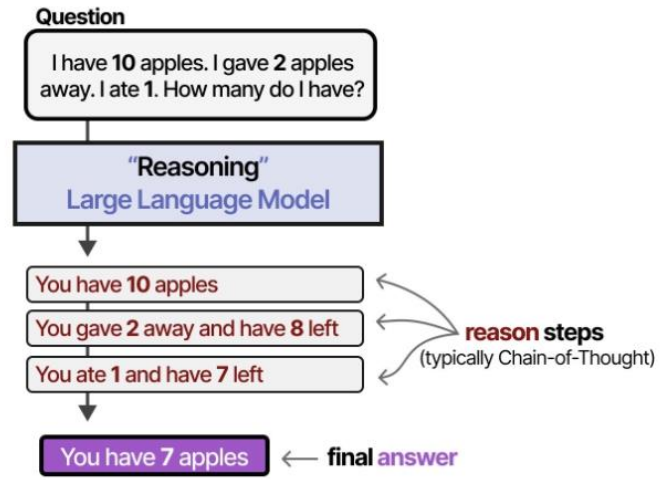
# Definition #3: AI Agents can *think*

## "Regular" LLMs

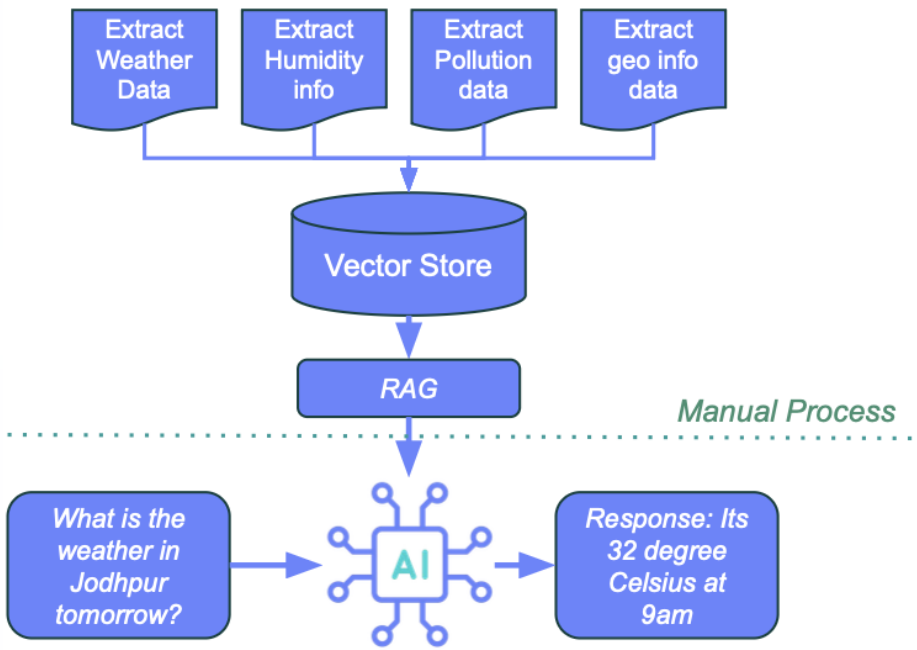


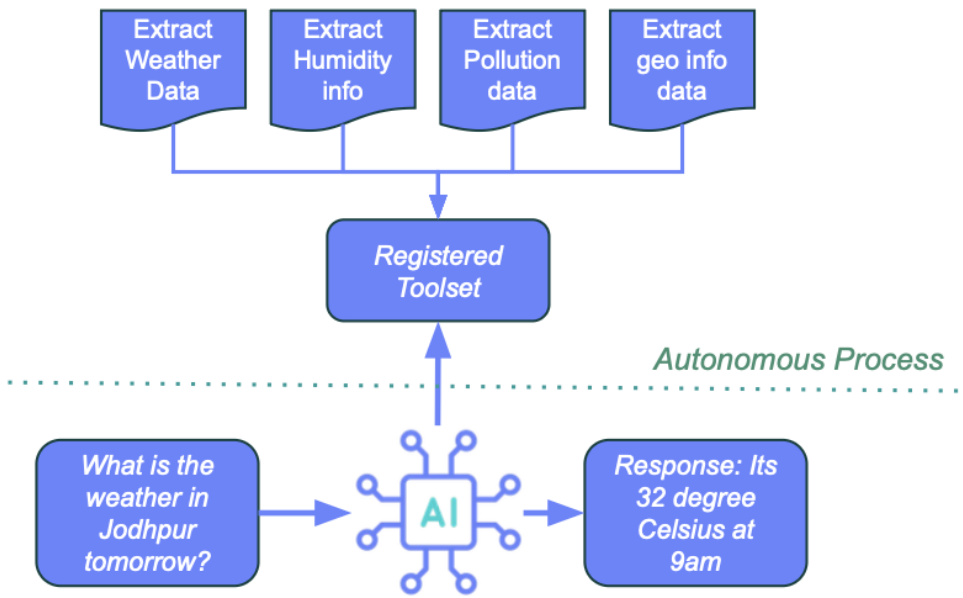

## "Reasoning" LLMs



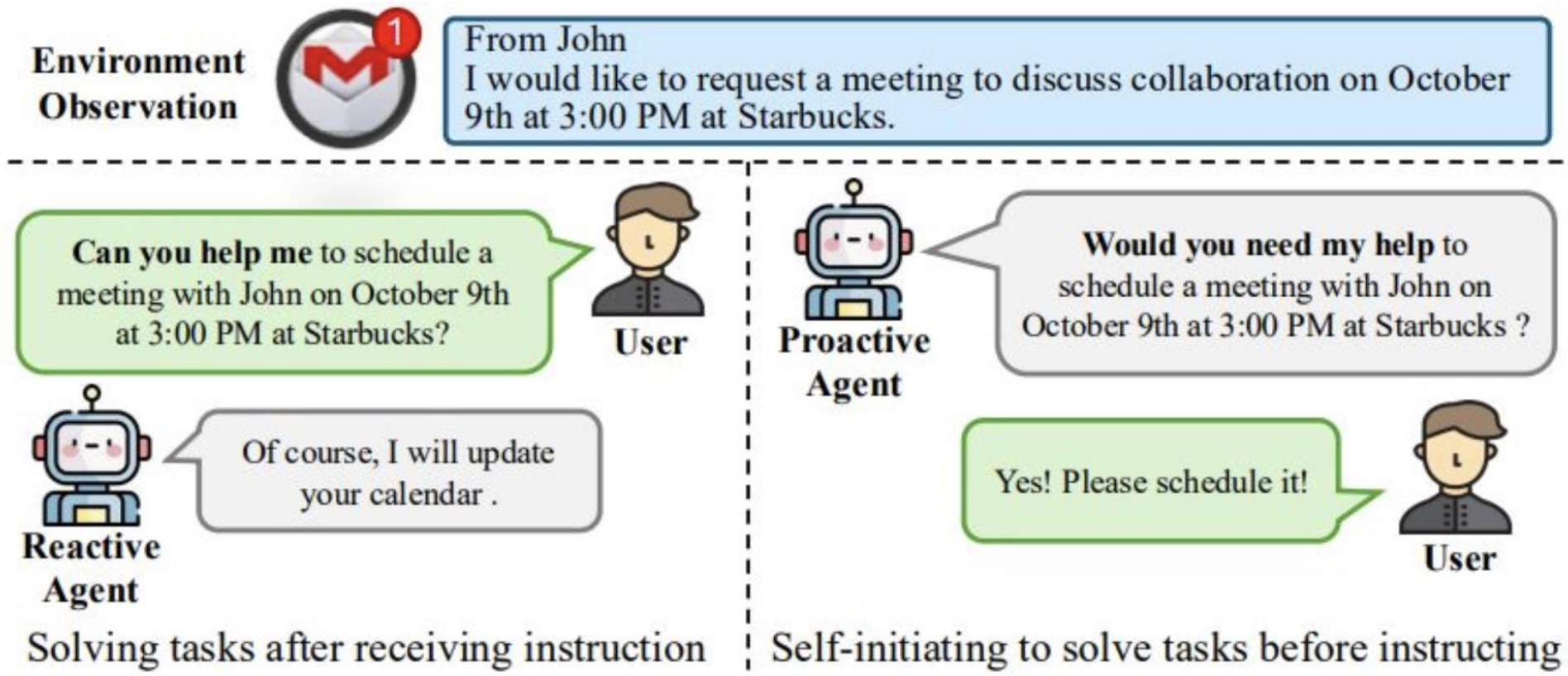



# Definition #4: AI Agents are *autonomous*

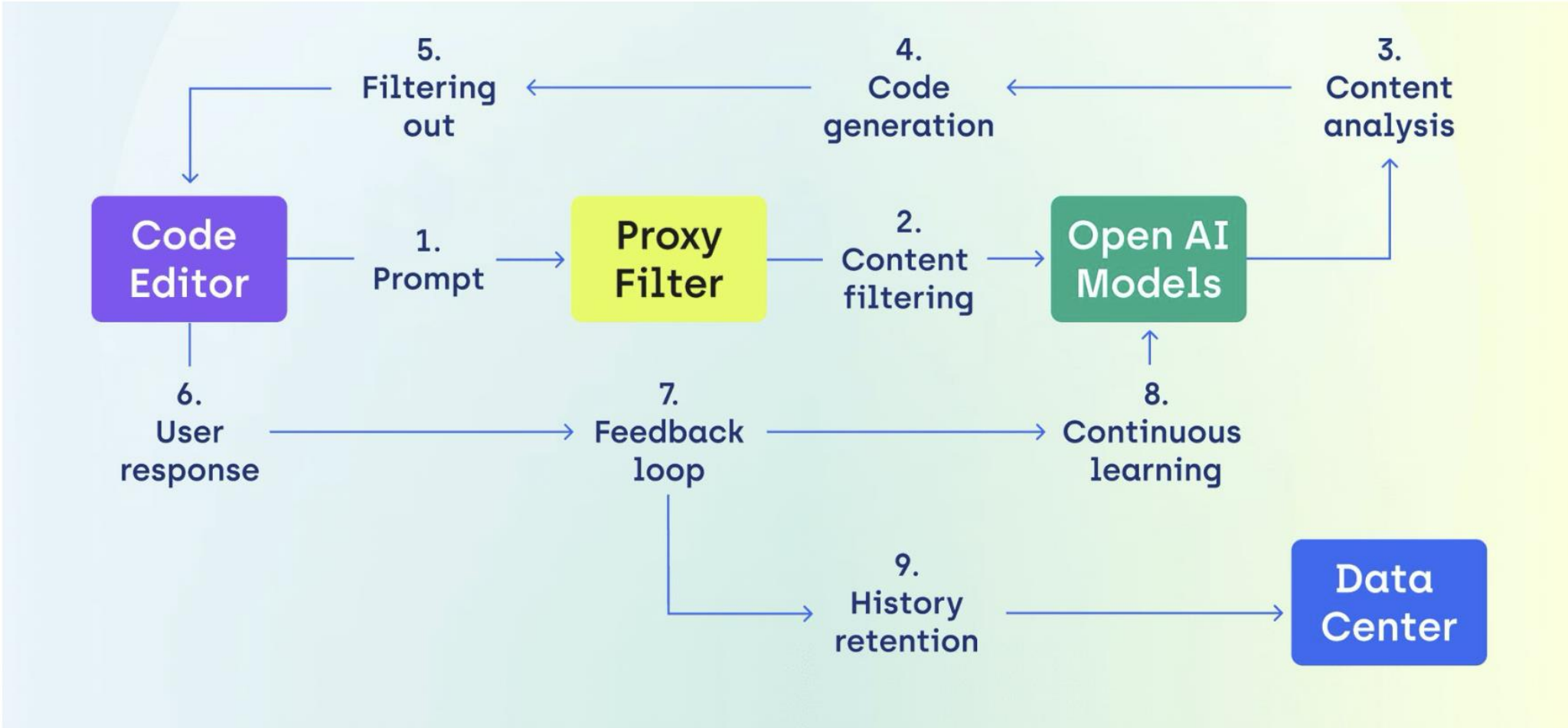


# Definition #5: AI Agents are *proactive*

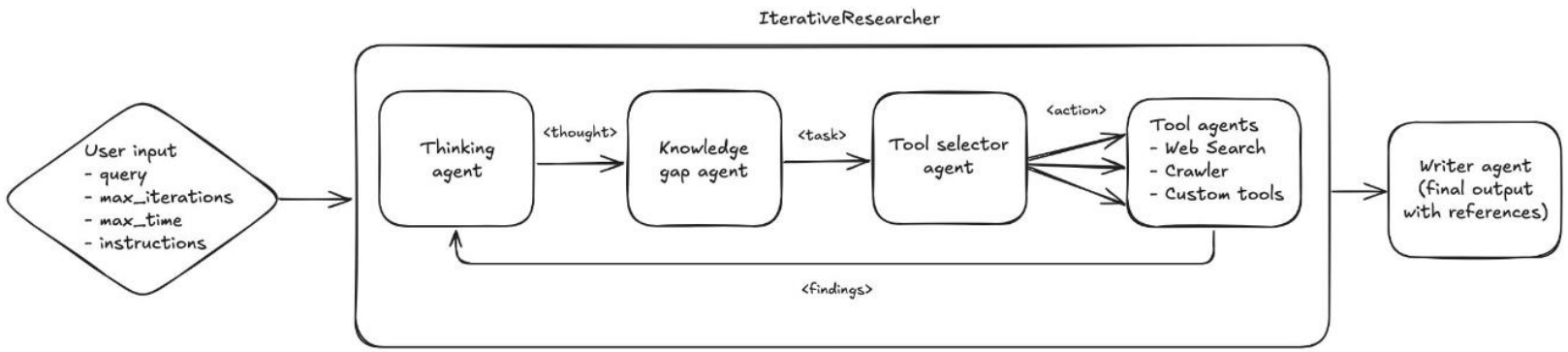


# Coding Agent: Github CoPilot

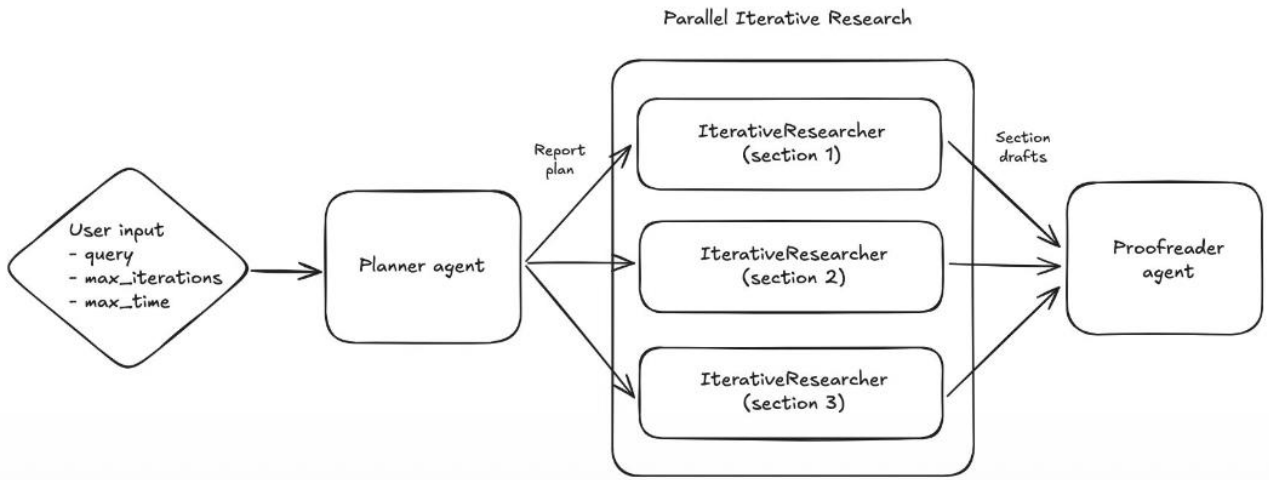


# Researcher Agent: Deep Research

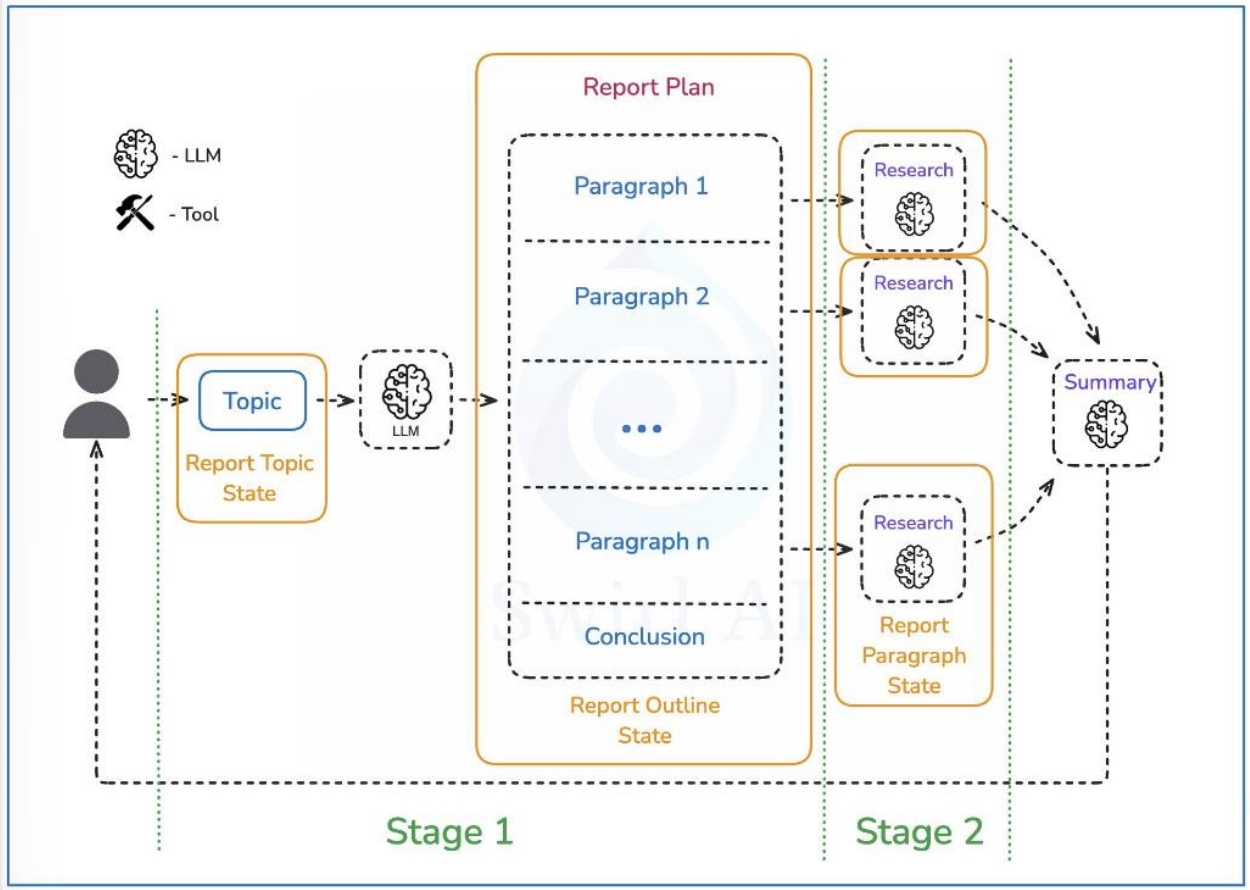
Mode 1: Simple

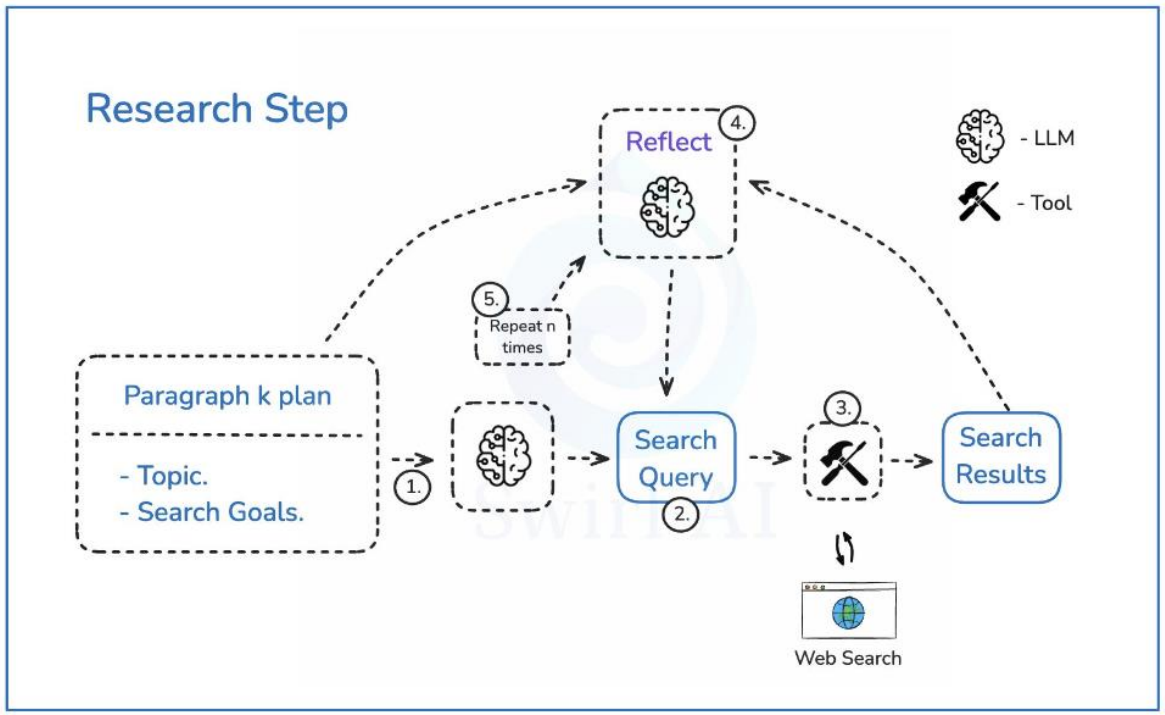



Mode 2: Deep

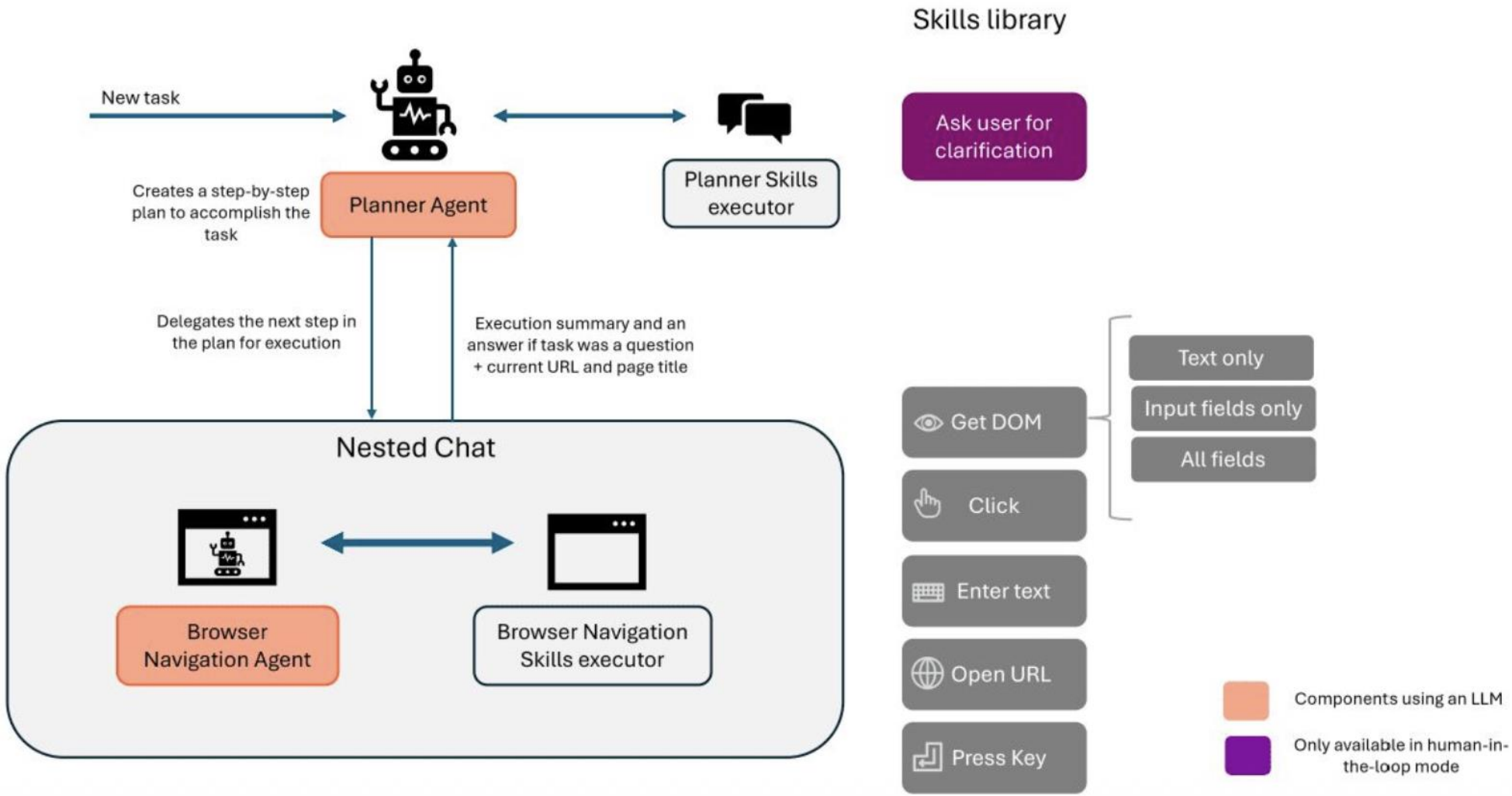


# Researcher Agent: Deep Research



# Web Agent: Browsing Internet



# What it's like as a computer use agent



Tool: Move(48, 53)
Tool: Click()

*Close your eyes for 3-5 seconds*




"You are a computer use agent.

You can move your mouse, click, type and use keys.

Your task:

Install spotify and login as Barry"

"You are a computer use agent.

You can move your mouse, click, type and use keys.

Your task:

Install spotify and login as Barry

You moved and clicked"

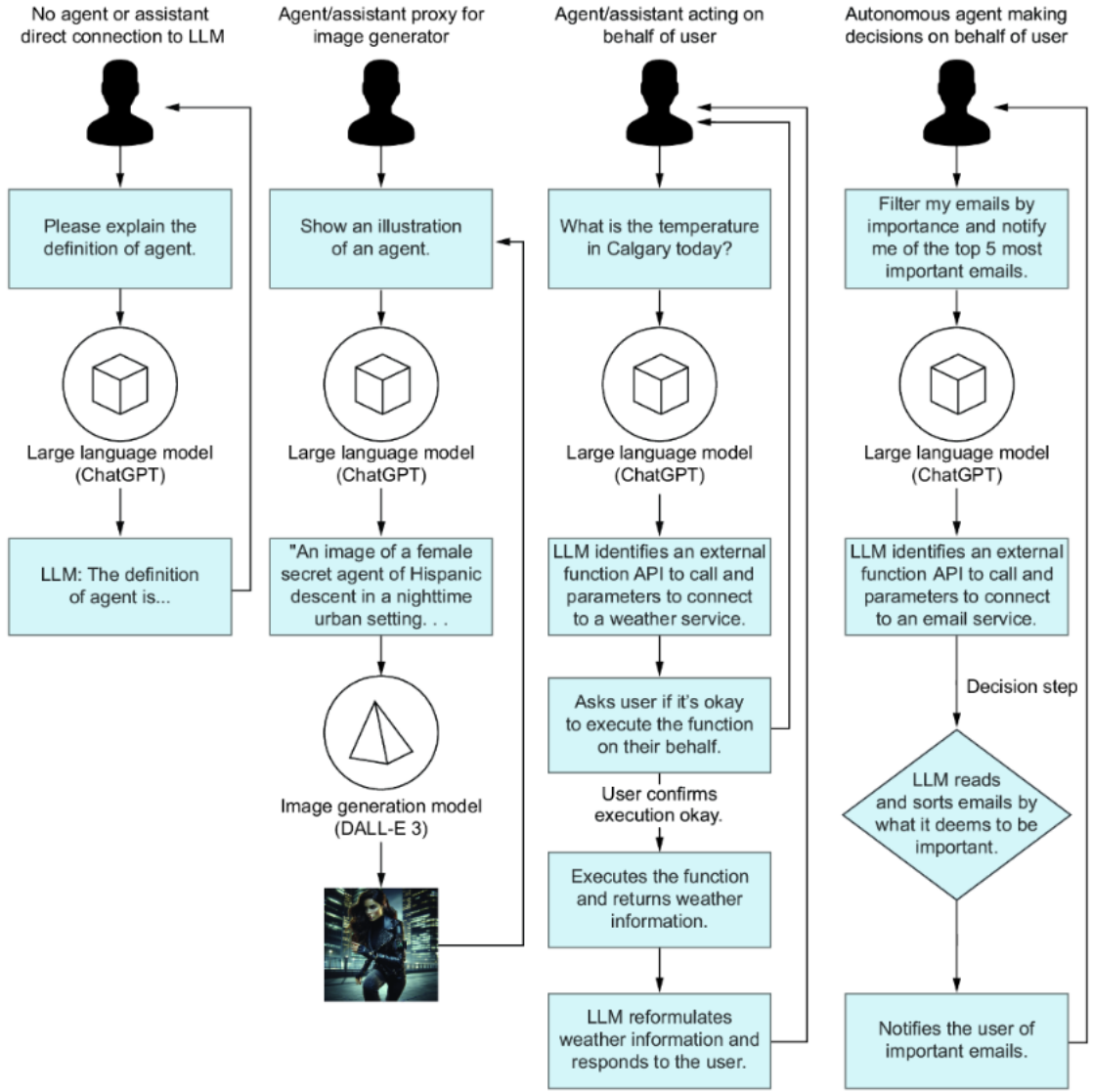
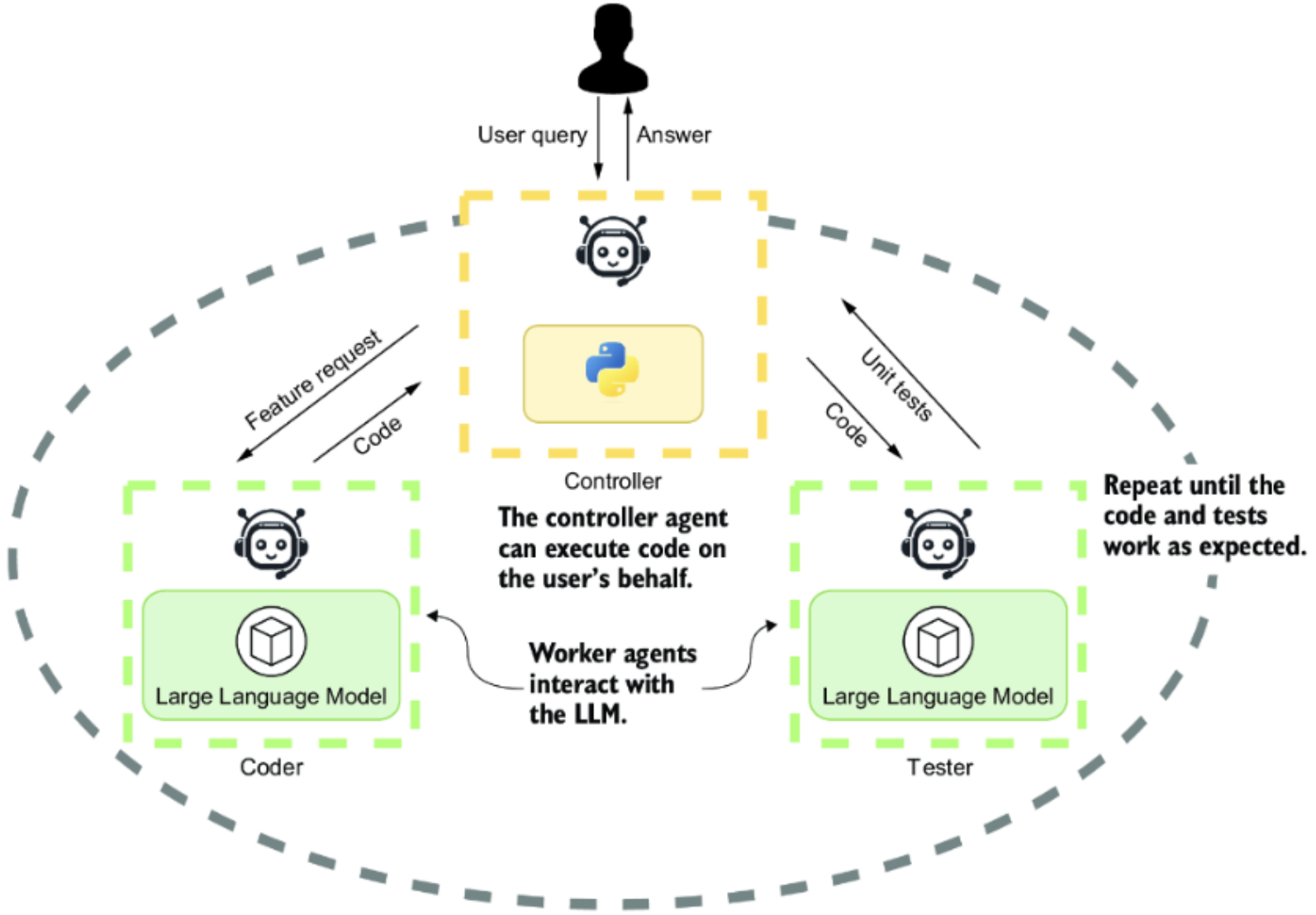
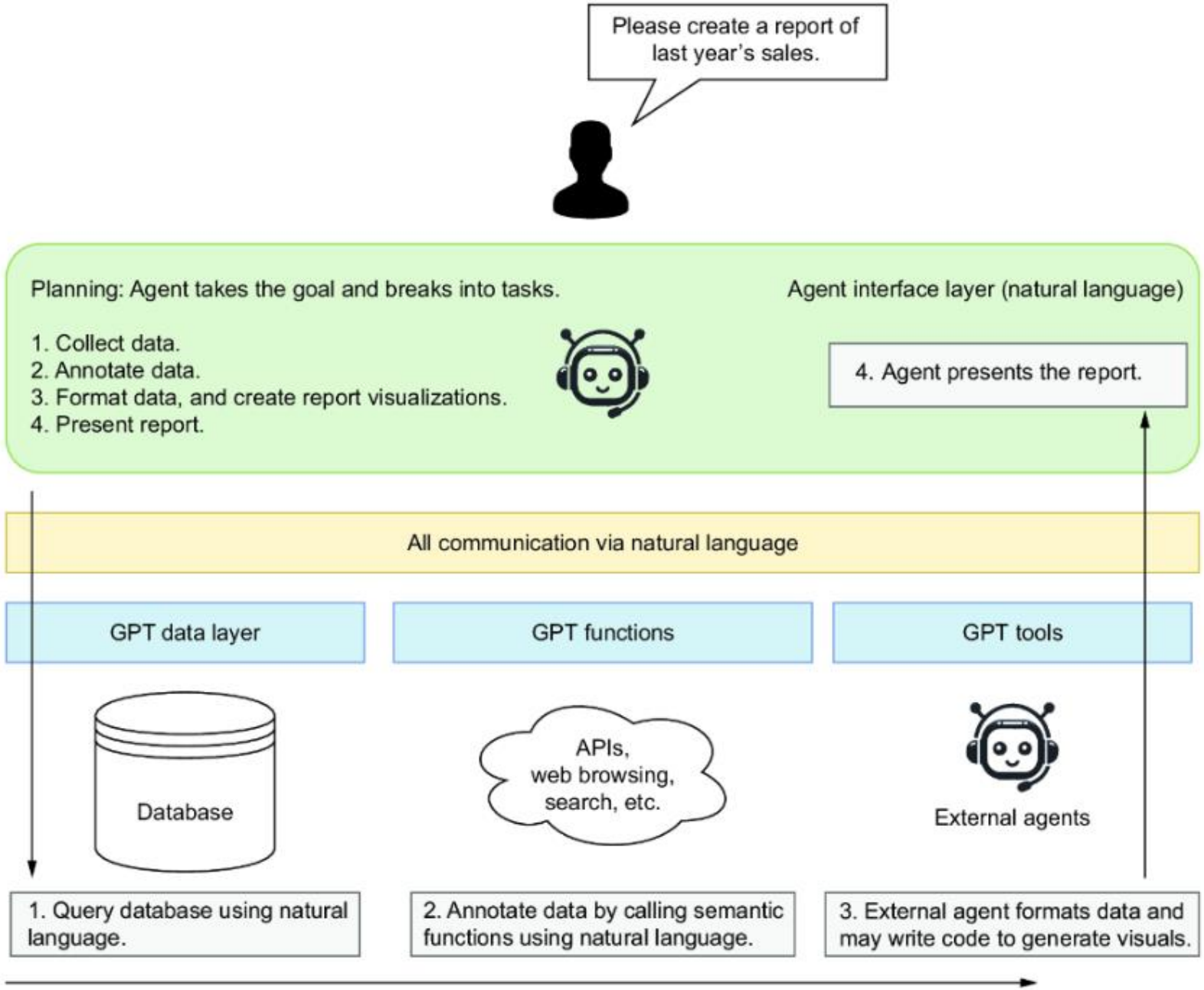


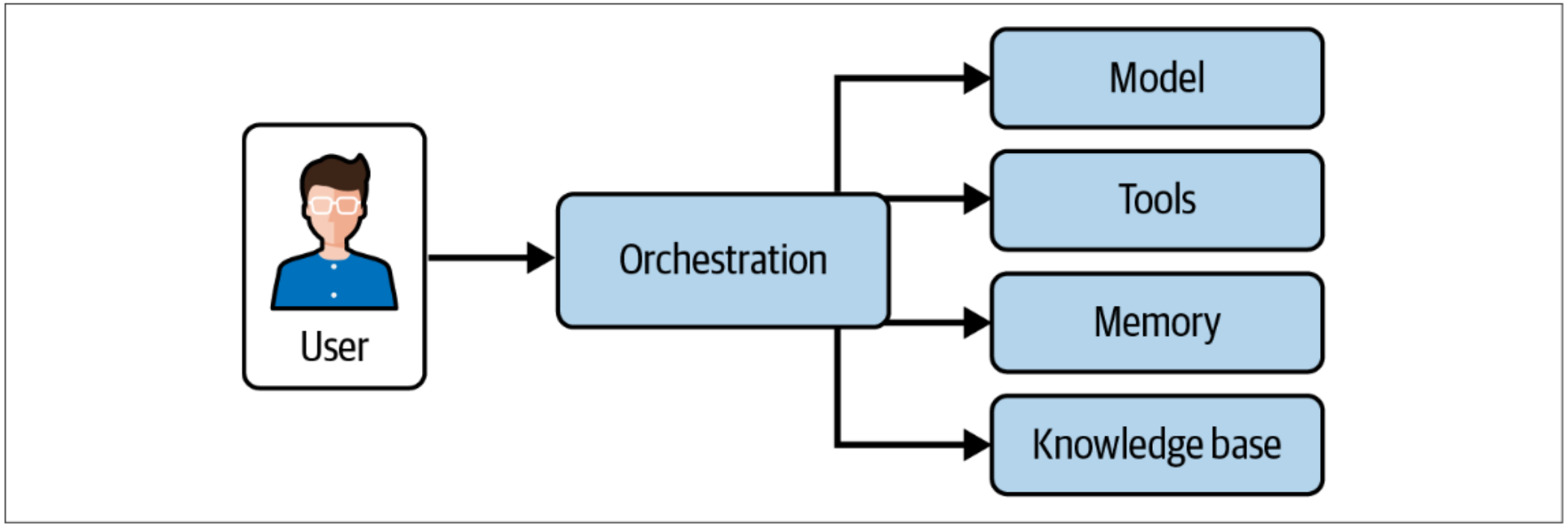

**Figure 1.1 The differences between the LLM interactions from direct action compared to using proxy agents, agents, and autonomous agents**

“In this example of a multi-agent system, the controller or agent proxy communicates directly with the user. Two agents—a coder and a tester—work in the background to create code and write unit tests to test the code.”




Please create a report of last year's sales.
Planning: Agent takes the goal and breaks into tasks.
1. Collect data.
2. Annotate data.
3. Format data, and create report visualizations.
4. Present report.
Agent interface layer (natural language)
4. Agent presents the report.
All communication via natural language
GPT data layer
GPT functions
GPT tools
Database
APIs, web browsing, search, etc.
External agents
1. Query database using natural language.
2. Annotate data by calling semantic functions using natural language.
3. External agent formats data and may write code to generate visuals.

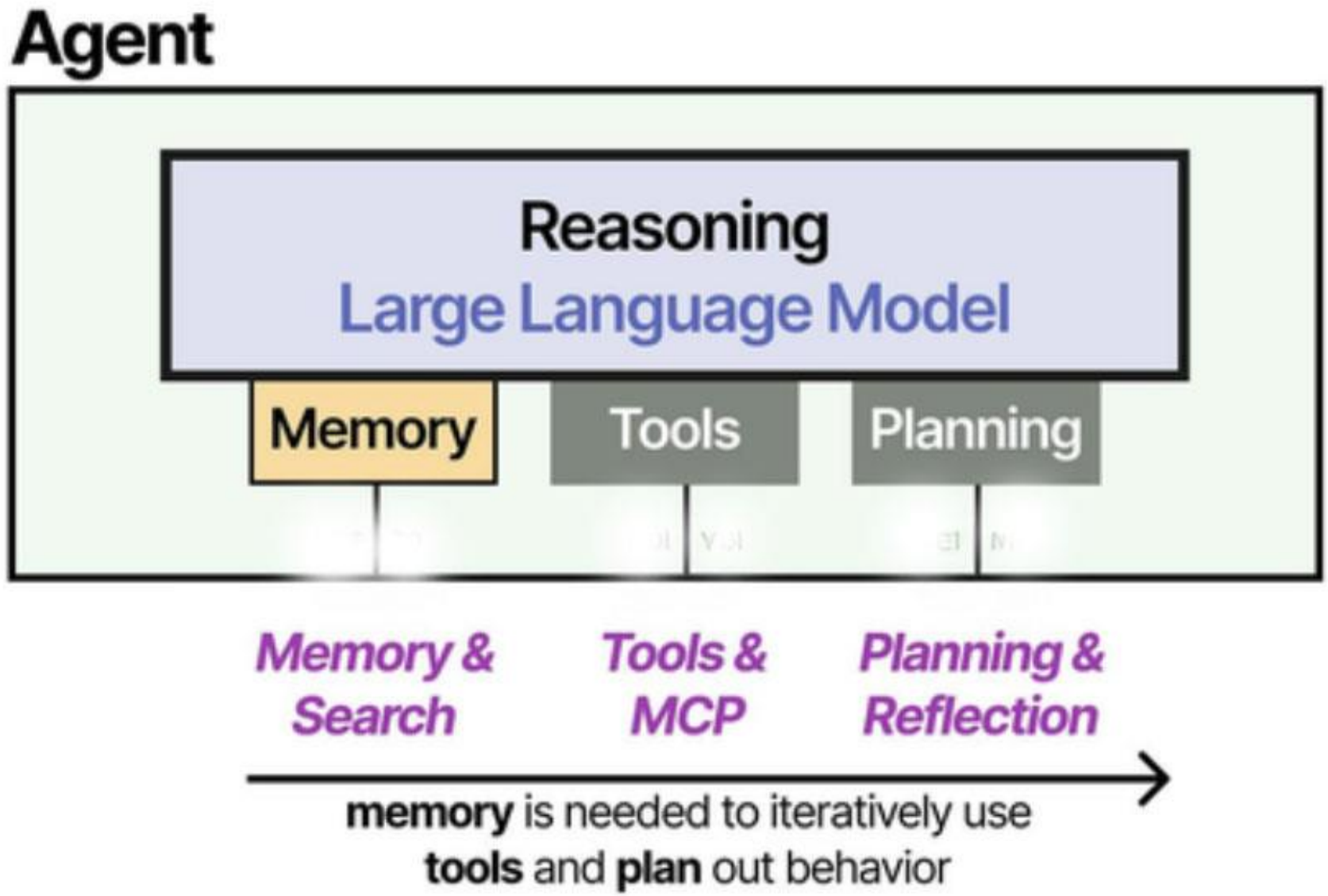
system.

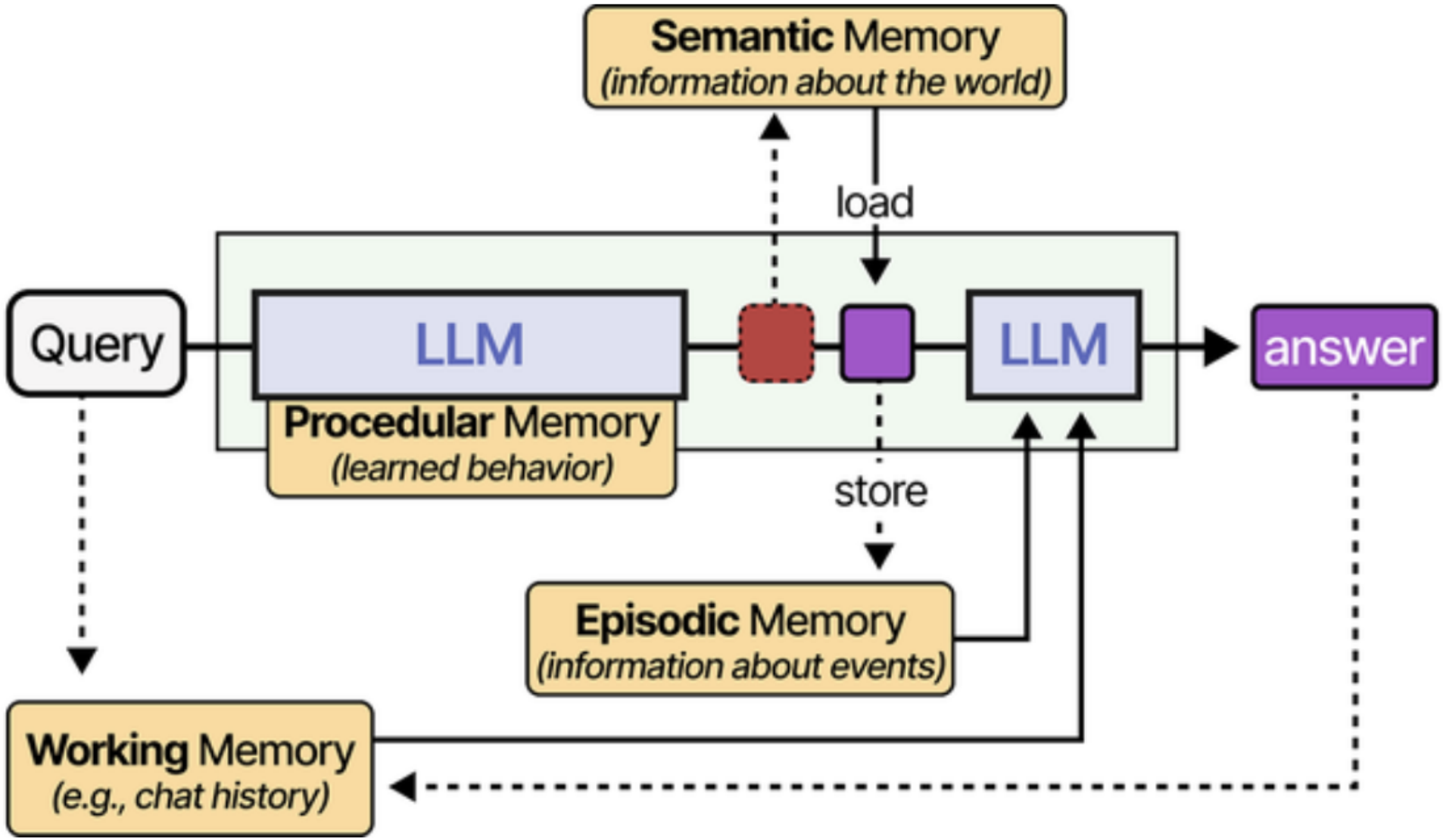


# Memory

Ashish Bansal


Without memory
Hi! I'm Maarten.
Hello Maarten! How can I help you?
LLM
next session
What is my name?
You have not given me your name yet.
LLM
forgetful behavior
(historical information is not stored)


Agent
Reasoning
Large Language Model
Memory
Tools
Planning
Memory & Search
Tools & MCP
Planning & Reflection
memory is needed to iteratively use tools and plan out behavior


Semantic Memory
(information about the world)
load
Query
LLM
LLM
answer
Procedular Memory
(learned behavior)
store
Episodic Memory
(information about events)
Working Memory
(e.g., chat history)

“**Working memory** is a type of short-term memory that is typically defined as a system with limited capacity that temporarily holds information that we need for things like decision-making and reasoning. For LLMs, it is typically data that persists across LLM calls. More specifically, it is the chat history of the LLM that is being stored in memory and continuously fed back to the LLM.”

*Semantic memory*

Involves remembering knowledge about the world (e.g., the capital of France). For agents, this may involve querying an external database like Wikipedia or the codebase that you are working on.

*Episodic memory*

Involves remembering specific events and experiences from one’s past (e.g., your last birthday party). For agents, this typically involves specific actions the agent has taken thus far and their outcomes.

*Procedural memory*

Involves remembering patterns of how to do things (e.g., writing code in Python). For agents, this can be information hidden in its parameters (also called parametric memory) or the system prompt, which persists across calls.

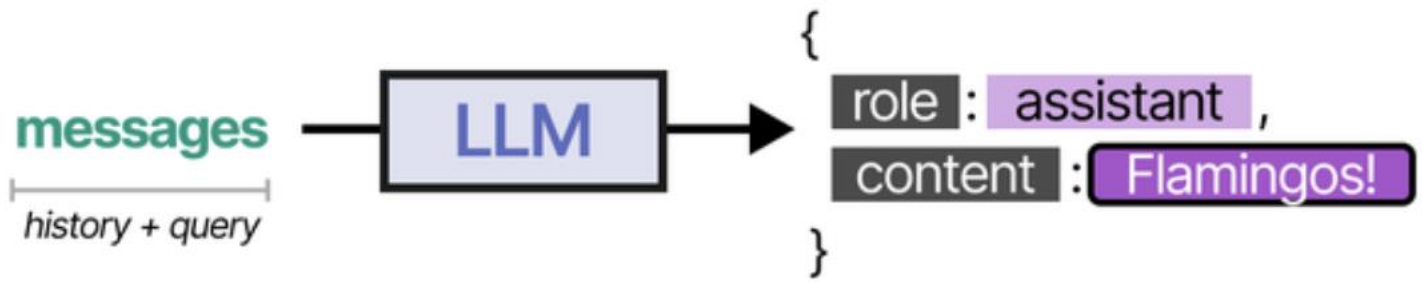
```
messages =
[
  { role: system , content : You are a helpful assistant. },
  { role: user , content : Flamingos are my favorite animals. },
  { role:  assistant , content : That's wonderful! },
  { role: user , content : Tell me something about flamingos. },
  { role:  assistant , content : Flamingos are a type of ... },
```

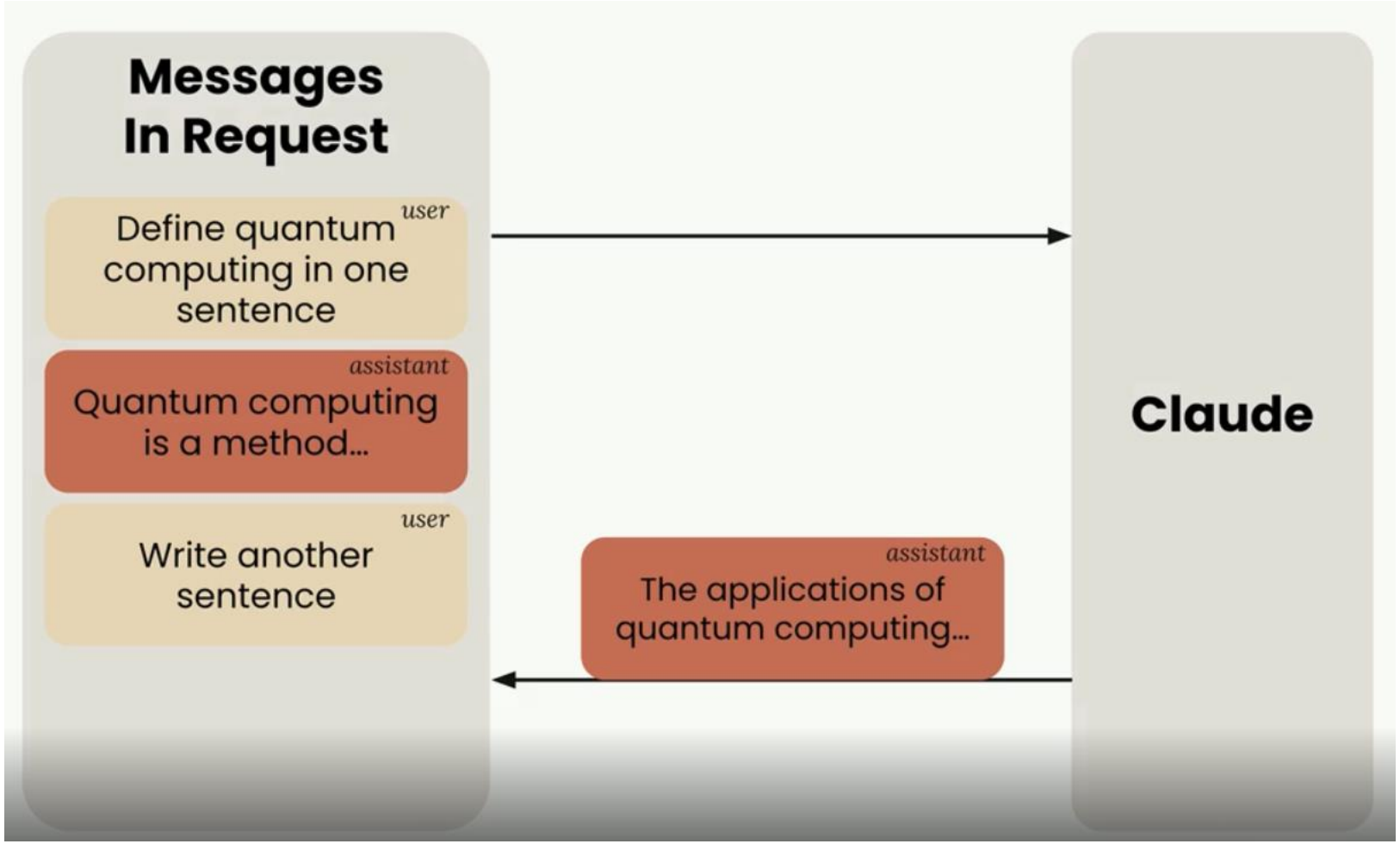
*conversation history*

```
  { role: user , content : What is my favorite animal? },
```

*query*

```
]
```




Messages In Request
Define quantum computing in one sentence
user
Quantum computing is a method...
assistant
Write another sentence
user
Claude
The applications of quantum computing...
assistant

```python
def add_user_message(messages, text):
    user_message = {"role": "user", "content": text}
    messages.append(user_message)

def add_assistant_message(messages, text):
    assistant_message = {"role": "assistant", "content": text}
    messages.append(assistant_message)

def chat(messages):
    message = client.messages.create(
        model=model,
        max_tokens=1000,
        messages=messages,
    )
    return message.content[0].text
```

```python
# Start with an empty message list
messages = []

# Add the initial user question
add_user_message(messages, "Define quantum computing in one sentence")

# Get Claude's response
answer = chat(messages)

# Add Claude's response to the conversation history
add_assistant_message(messages, answer)

# Add a follow-up question
add_user_message(messages, "Write another sentence")

# Get the follow-up response with full context
final_answer = chat(messages)
```

## Semantic Memory (Facts & Knowledge)

Semantic memory is the agent's encyclopedia. It stores individual pieces of knowledge or "facts." These can be independent strings, such as *"The user is a vegetarian,"* or structured attributes attached to an entity, like `{"food restrictions": "User is a vegetarian"}`. This is where the agent stores concepts and relationships regarding specific domains, people, or places.

The primary role of semantic memory is to provide a reliable source of truth. For an enterprise agent, this might involve storing internal company documents or technical manuals. For a personal assistant, semantic memory builds a persistent user profile. It recalls specific preferences like `{"music": "User likes rock music"}` or constraints like `{"dog": "User has a dog named George"}`. This allows the agent to retrieve relevant facts without searching through a noisy conversation history.

1. We define a helper function to add text to memory and categorize it.

```
def mem_add_text(text: str, category: str = “semantic”, **meta) -> str:
    “””Add a single text memory. No LLM is used for extraction or summarization.”””
    metadata = {“category”: category}
    for k, v in meta.items():
        if isinstance(v, (str, int, float, bool)) or v is None:
            metadata[k] = v
        else:
            metadata[k] = str(v)
    memory.add(text, user_id=MEM_USER_ID, metadata=metadata, infer=False)
    return f”Saved {category} memory.”
```

2. We insert specific facts about the user.

```
facts: list[str] = [
    “User prefers vegetarian meals.”,
    “User has a dog named George.”,
    “User is allergic to gluten.”,
    “User’s brother is named Mark and is a software engineer.”,
]
for f in facts:
    print(mem_add_text(f, category=”semantic”))
```

3. We can now search for this specific semantic information.

```
results = memory.search(“brother job”, user_id=MEM_USER_ID, limit=1)
print(results[“results”][0][“memory”])
```

4. It outputs:

```
User’s brother is named Mark and is a software engineer.
```

## Episodic Memory (Experiences & History)

Episodic memory is the agent's personal diary. It records past interactions, but unlike timeless facts, these memories have a timestamp. It captures *"what happened and when."*

This memory type is essential for maintaining conversational context and understanding relationship dynamics. A semantic fact might be *"User is frustrated with his brother."* An episodic memory would be: *"On Tuesday, the user expressed frustration that their brother, Mark, always forgets their birthday. I provided an empathetic response. [created_at=2025-08-25]."*

This "episode" provides nuanced context. If the topic comes up again, the agent can say, "I know the topic of your brother's birthday can be sensitive," rather than just stating a fact. It also allows the agent to answer questions like *"What happened last week?"* [7].

1. We define a dialogue and ask the LLM to summarize it into an episode.

```
dialogue = [
    {"role": "user", "content": "I'm stressed about my project deadline on Friday."},
    {"role": "assistant", "content": "I'm here to help—what's the blocker?"},
    {"role": "user", "content": "Mainly testing. I also prefer working at night."},
    {"role": "assistant", "content": "Okay, we can split testing into two sessi
]

episodic_prompt = f"""Summarize the following 3–4 turns as one concise 'episode
sentences).
Keep salient details and tone.

{dialogue}
"""
episode_summary = client.models.generate_content(model=MODEL_ID,
contents=episodic_prompt)
episode = episode_summary.text.strip()
```

2. We save this summary as an episodic memory.

```
print(
    mem_add_text(
        episode,
        category="episodic",
        summarized=True,
        turns=4,
    )
)
```

3. We can search for the "experience" later.

```
hits = mem_search("deadline stress", limit=1, category="episodic")
for h in hits:
    print(f"{h['memory']}\n")
```

4. It outputs:

```
A user, stressed about a Friday project deadline because of testing and a preference
for working at night, is advised to split the testing work into two manageable
sessions.
```

## Procedural Memory (Skills & How-To)

Procedural memory is the agent's muscle memory. It consists of skills, learned workflows, and "how-to" knowledge. It dictates the agent's ability to perform multi-step tasks.

This memory is often baked into the agent's system prompt as reusable tools or defined sequences. For example, an agent might store a `MonthlyReportIntent` procedure. When a user asks for a report, the agent retrieves this procedure: 1) Query sales DB, 2) Summarize findings, 3) Email user. This makes behavior reliable and predictable. It encodes successful workflows so the agent doesn't have to reason from scratch every time [8].

1. We define a procedure as a text block containing steps.

```
procedure_name = “monthly_report”
steps = [
    “Query sales DB for the last 30 days.”,
    “Summarize top 5 insights.”,
    “Ask user whether to email or display.”,
]
procedure_text = f”Procedure: {procedure_name}\nSteps:\n” + “\n”.join(f”{i + 1}. {s}”
for i, s in enumerate(steps))

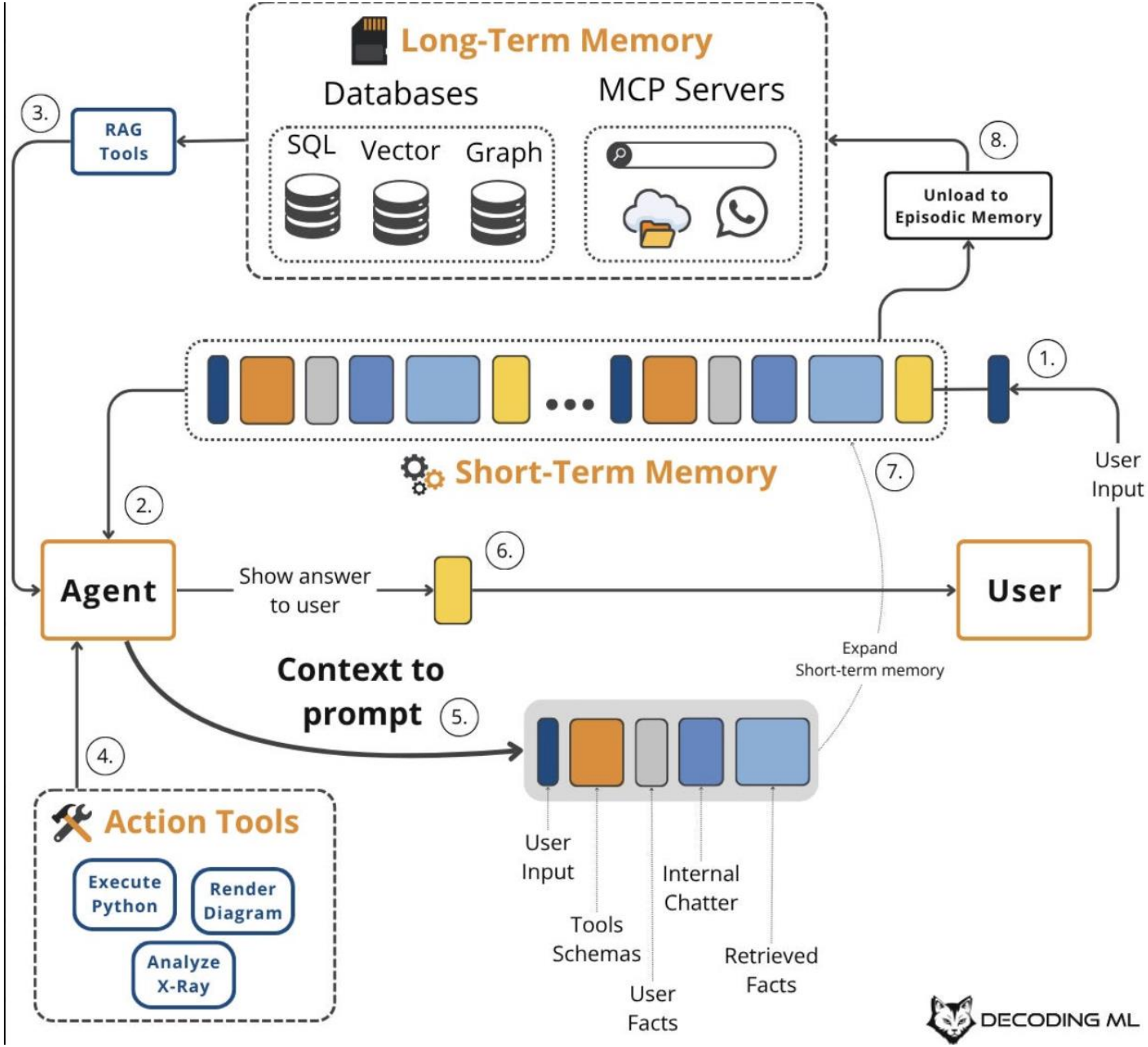
mem_add_text(procedure_text, category=”procedure”, procedure_name=procedure_name)
```

2. We retrieve the procedure by intent.

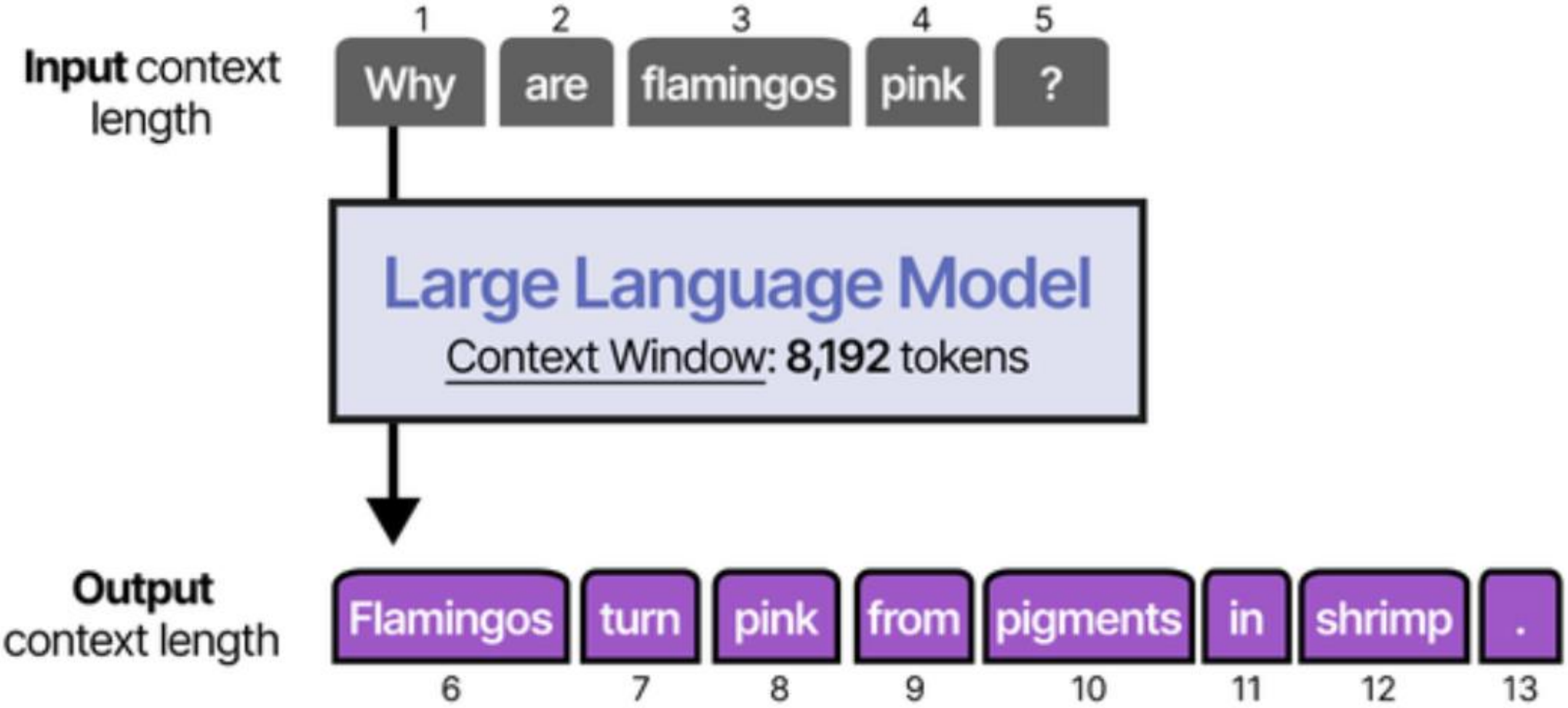
```
results = mem_search(”how to create a monthly report”, category=”procedure”, limit=1)
if results:
    print(results[0][”memory”])
```

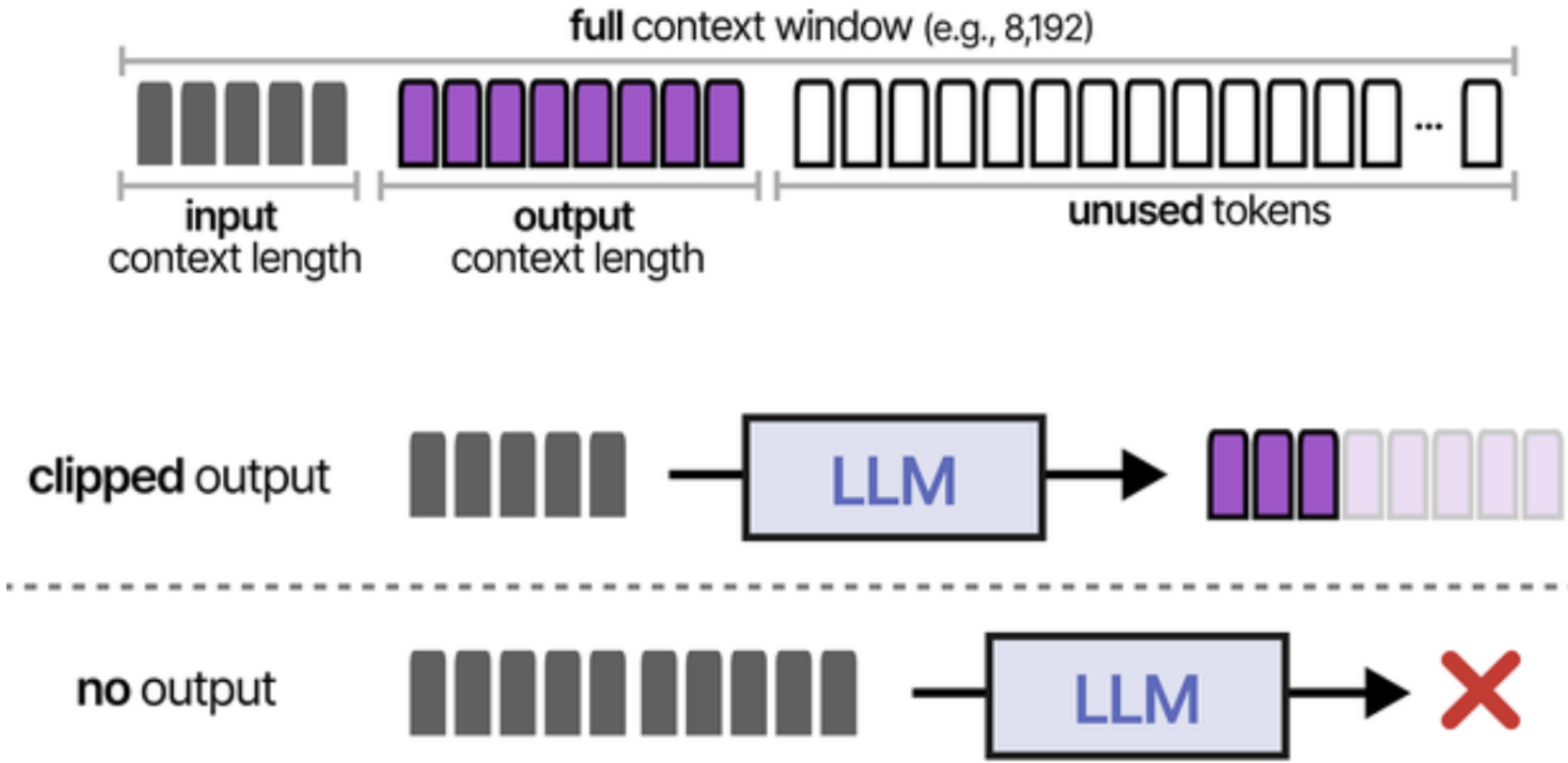
3. It outputs:

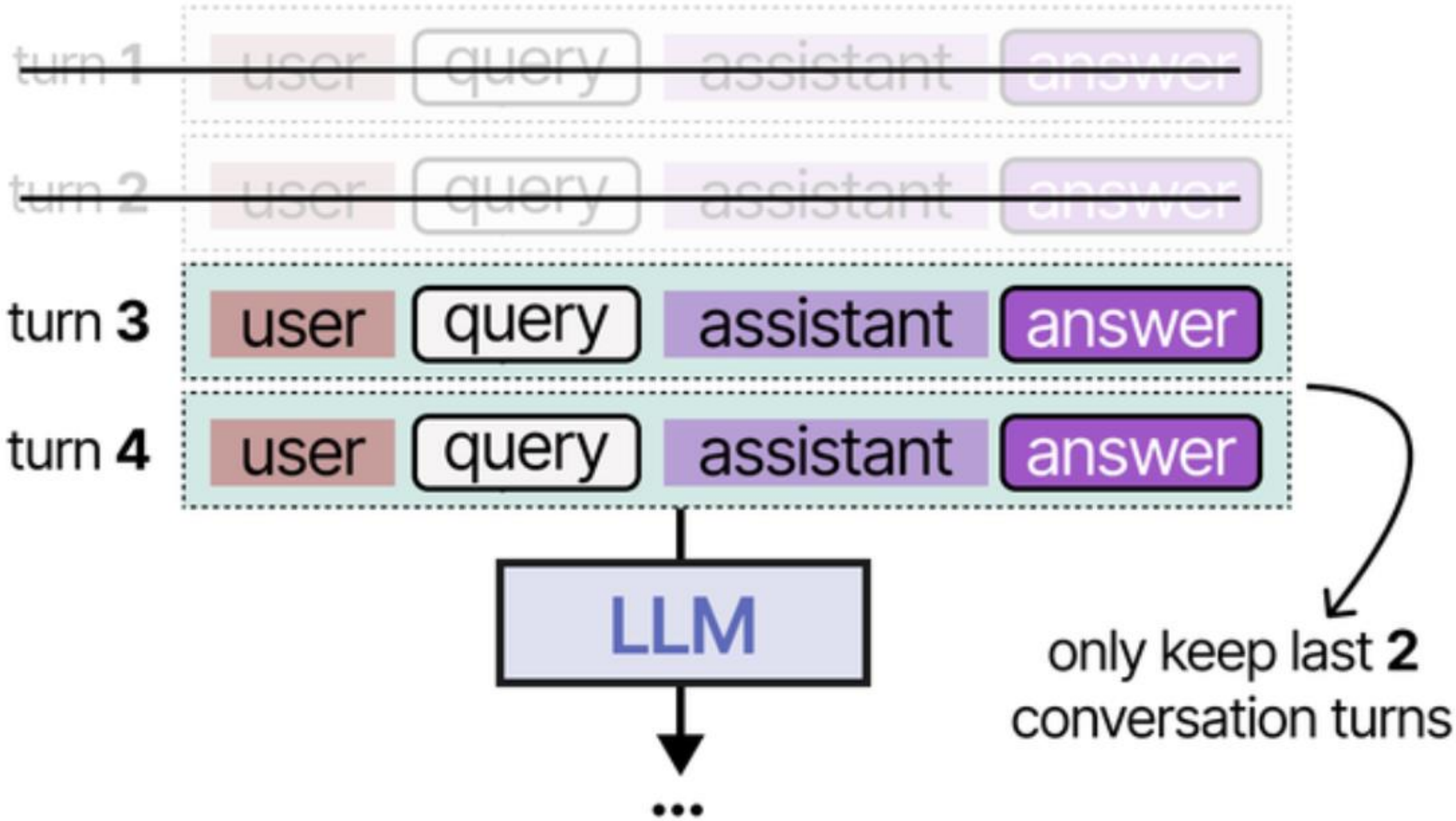
```
Procedure: monthly_report
Steps:
1. Query sales DB for the last 30 days.
2. Summarize top 5 insights.
3. Ask user whether to email or display.
```

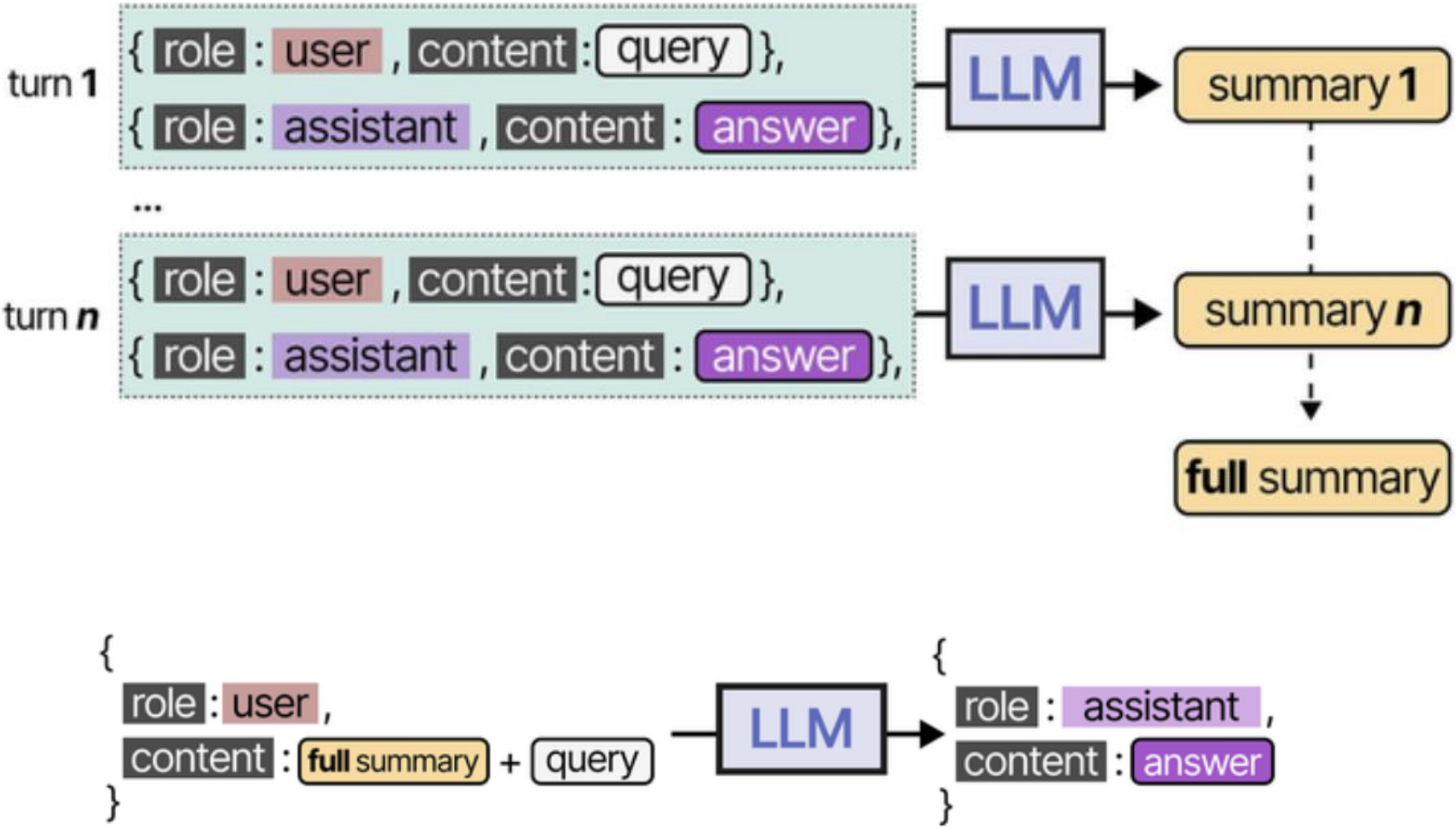

Long-Term Memory
Databases
MCP Servers
SQL
Vector
Graph
3.
RAG Tools
8.
Unload to Episodic Memory
1.
Short-Term Memory
7.
User Input
2.
Agent
Show answer to user
6.
User
Context to prompt
5.
Expand Short-term memory
4.
Action Tools
Execute Python
Render Diagram
Analyze X-Ray
User Input
Tools Schemas
User Facts
Internal Chatter
Retrieved Facts
DECODING ML

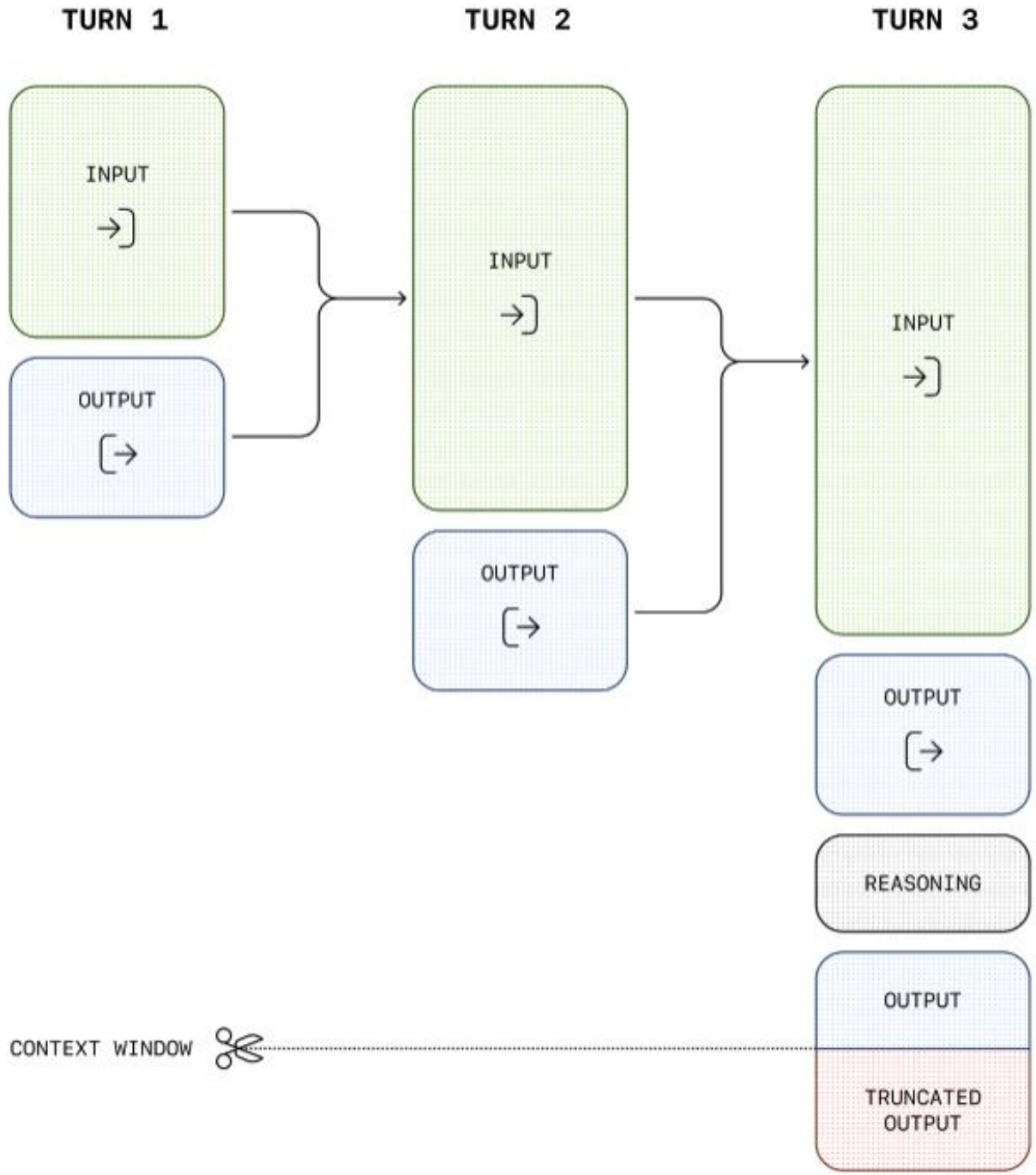
# Context Window




full context window (e.g., 8,192)
input
context length
output
context length
unused tokens
clipped output
LLM
no output
LLM


turn 1
user
query
assistant
answer
turn 2
user
query
assistant
answer
turn 3
user
query
assistant
answer
turn 4
user
query
assistant
answer
LLM
...
only keep last 2
conversation turns


turn 1
{ role : user , content : query },
{ role : assistant , content : answer },
...
turn n
{ role : user , content : query },
{ role : assistant , content : answer },
LLM
summary 1
LLM
summary n
full summary
{
role : user ,
content : full summary + query
}
LLM
{
role : assistant ,
content : answer
}


TURN 1
TURN 2
TURN 3
INPUT
OUTPUT
INPUT
OUTPUT
INPUT
OUTPUT
REASONING
OUTPUT
TRUNCATED OUTPUT
CONTEXT WINDOW

- **Output tokens** are the tokens generated by a model in response to a prompt. Each model has different limits for output tokens. For example, `gpt-4o-2024-08-06` can generate a maximum of 16,384 output tokens.
- A **context window** describes the total tokens that can be used for both input and output tokens (and for some models, reasoning tokens). Compare the context window limits of our models. For example, `gpt-4o-2024-08-06` has a total context window of 128k tokens.

Early Transformer-based models like the original BERT were limited to 512 tokens (roughly 380 words), which was sufficient for sentence-level tasks but inadequate for document-level understanding. GPT-3 increased this to 2,048 tokens, and subsequent models have pushed the boundaries dramatically: GPT-4 Turbo supports 128,000 tokens, Claude 3 handles up to 200,000 tokens, and Gemini 1.5 Pro can process up to 1 million tokens, which is enough to encompass entire books or lengthy codebases.

# Production Implementation Challenges

First is the **context window challenge**. Even with massive context windows, this space is a finite and expensive resource. The self-attention mechanism, central to LLMs, imposes quadratic computational and memory overhead

This leads to **information overload**, also known as context decay or the "lost-in-the-middle" problem. Research shows that as you stuff more information into the context, models lose their ability to focus on critical details

Another subtle issue is **context drift**, where conflicting versions of the truth accumulate over time. For example, if the memory contains both "The user's budget is $500" and later "The user's budget is $1,000," the agent can get confused. Without a mechanism to resolve or prune outdated facts, the agent's knowledge base becomes unreliable.

Finally, there is **tool confusion**. We often see failure when we provide an agent with too many tools, especially with poorly written descriptions or overlapping functionalities. The Gorilla benchmark shows that nearly all models perform worse when given more than one tool

# Key Strategies for Context Optimization

**Selecting the right context** is your first line of defense. Avoid providing all available context; instead, use RAG with reranking to retrieve only the most relevant facts.
Structured outputs can also ensure the LLM breaks responses into logical parts, passing only necessary pieces downstream. This dynamic context optimization filters content and selects critical information to maximize density within the limited context window

**Context compression** is crucial for managing long-running conversations. As message history grows, summarize or condense it to avoid overflowing the context window, much like managing your computer's RAM.

**Isolating context** involves splitting a complex problem across multiple specialized agents. Each agent maintains its own focused context window, preventing interference and improving performance

**Context Ordering** Place critical instructions at the start and the most recent or relevant data at the end.

**Format optimization**

Always use YAML instead of JSON, as it’s 66% more token-efficient

Let's walk through a concrete example. Imagine a user asks a healthcare assistant: `I have a headache. What can I do to stop it? I would prefer not to take any medicine.`

Before the LLM even sees this query, a context engineering system gets to work:

1. It retrieves the user's patient history, known allergies, and lifestyle habits from an **episodic memory** store, often a vector or graph database
2. It queries a **semantic memory** of up-to-date medical literature for non-medicinal headache remedies
3. It assembles this information, along with the user's query and the conversation history, into a structured prompt.
4. We send the prompt to the LLM, which generates a personalized, safe, and relevant recommendation.
5. We log the interaction and save any new preferences back to the user's episodic memory.

```
<INSTRUCTIONS>
1. Analyze the user's query and the provided context.
2. Use the patient history to understand their health profile and preferences.
3. Use the retrieved medical knowledge to form your recommendation.
4. If you lack sufficient information, ask clarifying questions.
5. Always prioritize safety and advise consulting a doctor for serious issues.
</INSTRUCTIONS>

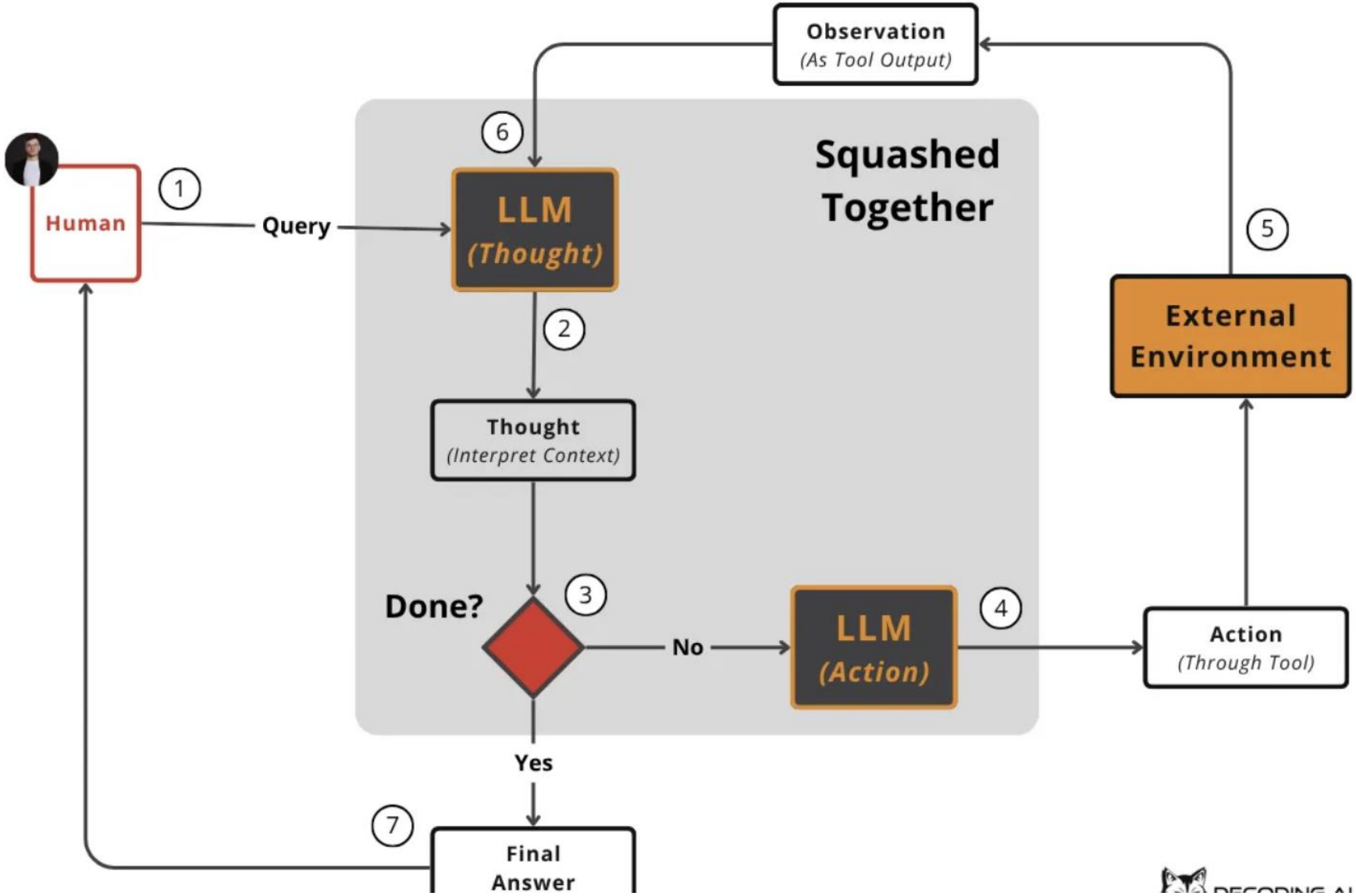
<PATIENT_HISTORY>
{retrieved_patient_history}
</PATIENT_HISTORY>

<MEDICAL_KNOWLEDGE>
{retrieved_medical_articles}
</MEDICAL_KNOWLEDGE>

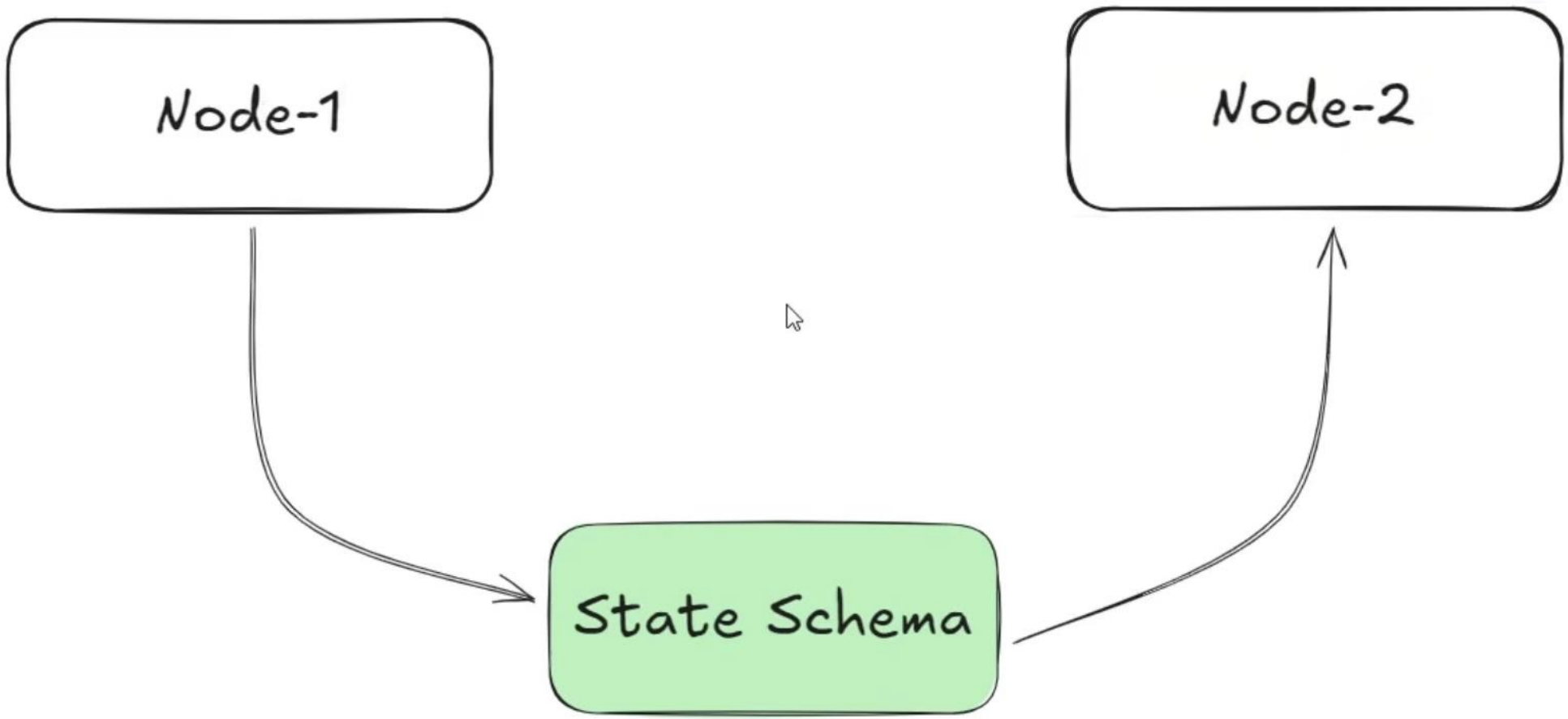
<CONVERSATION_HISTORY>
{formatted_chat_history}
</CONVERSATION_HISTORY>

<USER_QUERY>
{user_query}
</USER_QUERY>

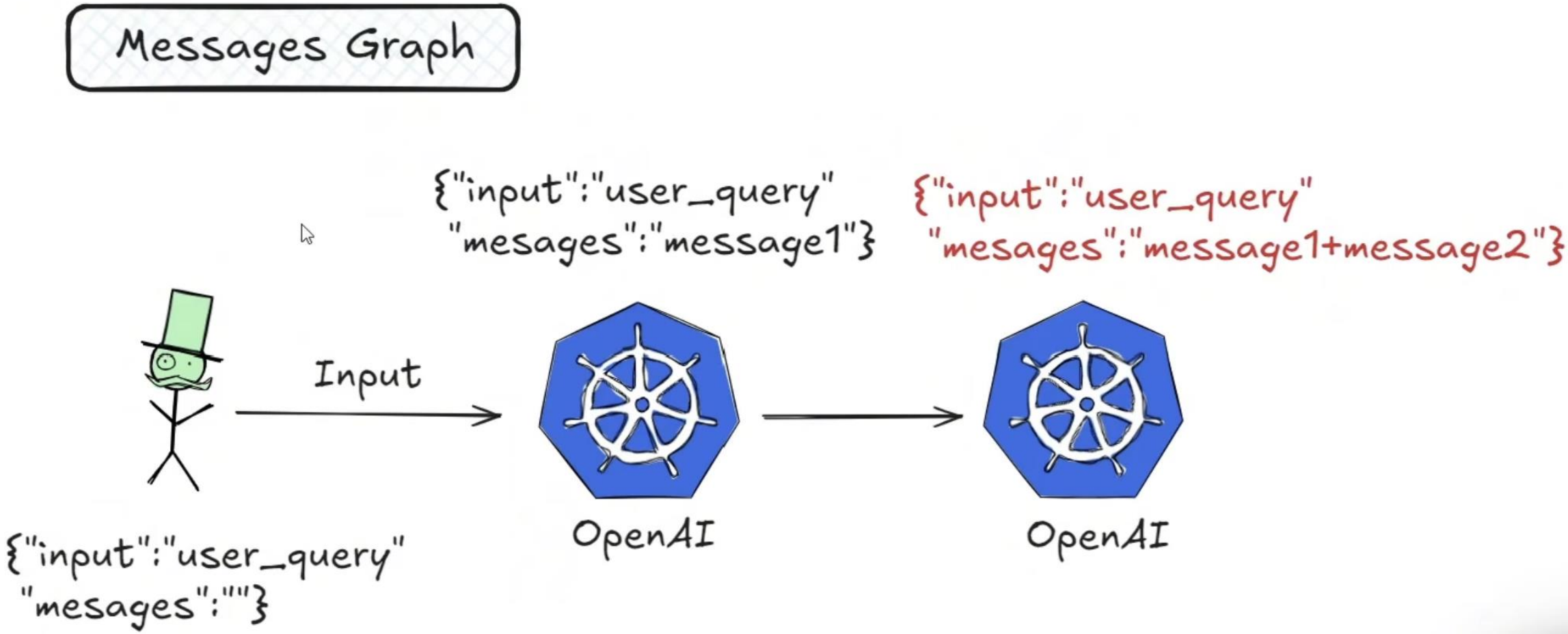
Based on all the information above, provide a helpful response.
"""
```


Observation
(As Tool Output)
Squashed
Together
6
Human
1
Query
LLM
(Thought)
5
External
Environment
2
Thought
(Interpret Context)
Done?
3
No
LLM
(Action)
4
Action
(Through Tool)
Yes
7
Final
Answer

In Langchain, the output of one node is the input of the next node but, in LangGraph, it's not the case



# Messages Graph



# Messages Graph



Input

```
{
 "mesages":[HumanMessage("somthing")]
}
```



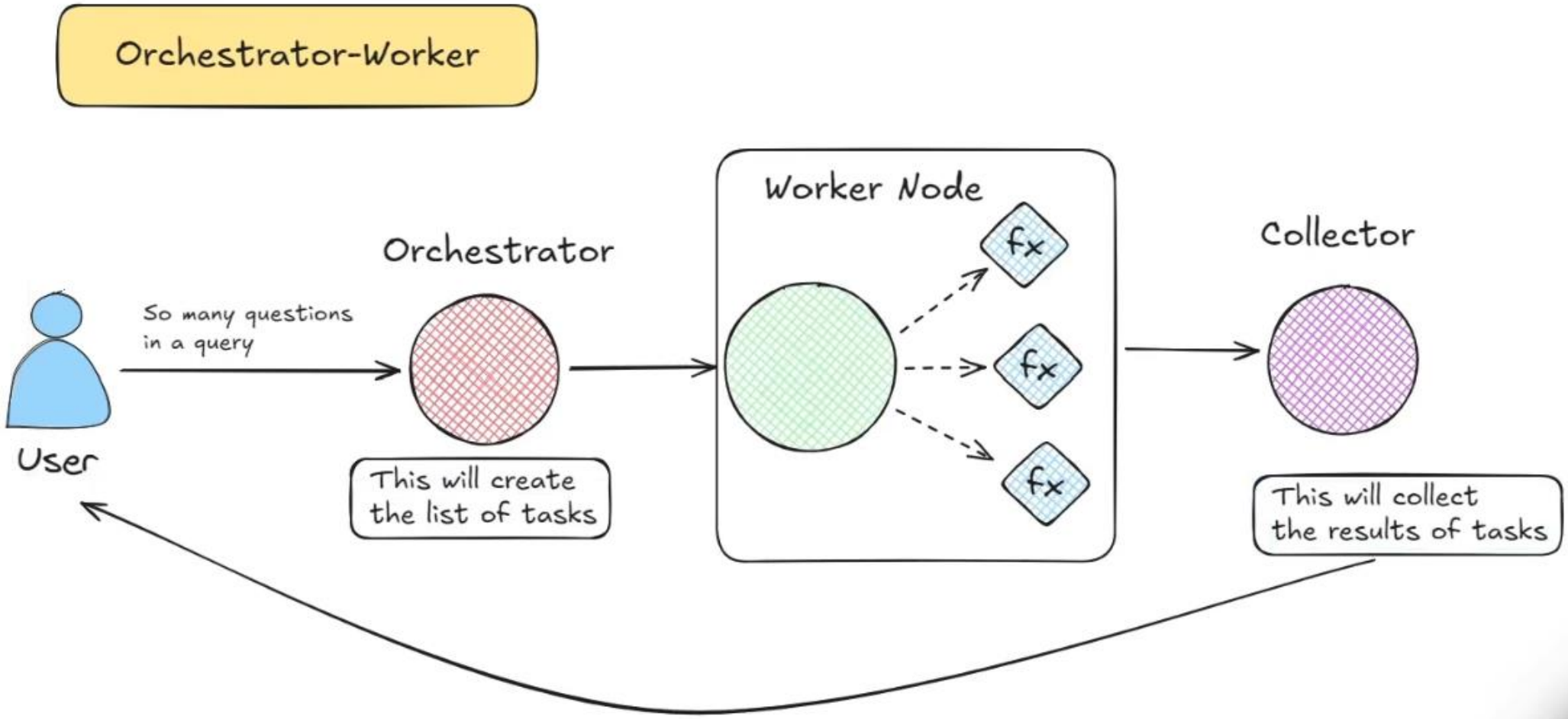
OpenAI

```
{
 "mesages":[HumanMessage("somthing"),
            AIMessage("something") ]
}
```



OpenAI

```
{
 "mesages":[HumanMessage("somthing"),
            AIMessage("something"),
            AIMessage("something")]
}
```


Orchestrator-Worker
Worker Node
Orchestrator
Collector
fx
fx
fx
So many questions
in a query
User
This will create
the list of tasks
This will collect
the results of tasks

# Generator - Evaluator

